

第三课

卷积神经网络主干网

复旦大学可信具身智能研究院

陈智能

2026-9-21

课程概要

卷积神经网络发展概述（to ResNet）

卷积神经网络发展概述（after ResNet）

Transformer及典型网络架构

Part 1

卷积神经网络发展概述（to ResNet)

卷积神经网络结构

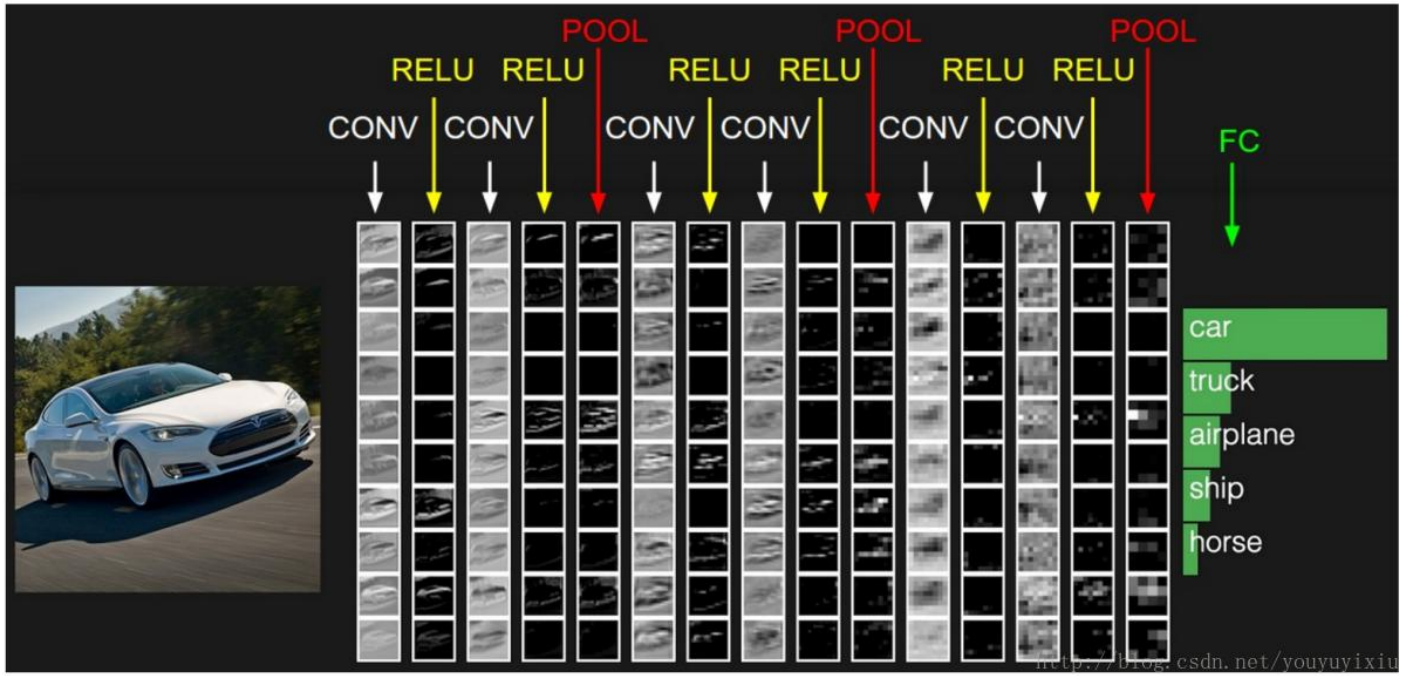
卷积神经网络的一般结构

- 提取特征：

  卷积层+激活层 和 池化层 的组合多次出现

- 分类器/检测器/分割器：

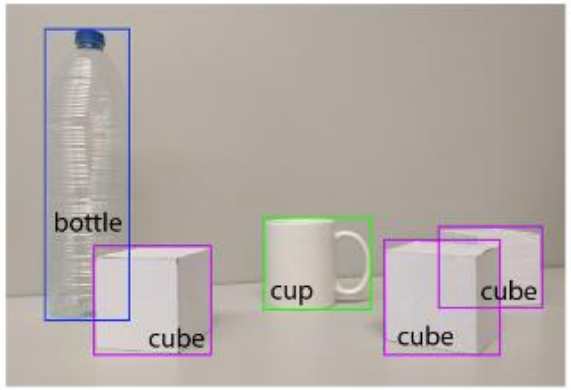
  多个全连接 或 特殊的CNN结构作为输出层



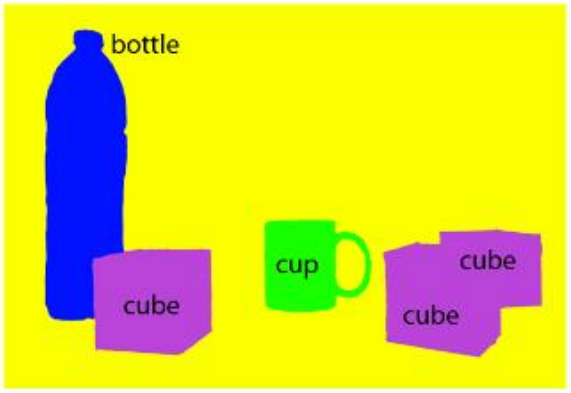
计算机视觉典型任务



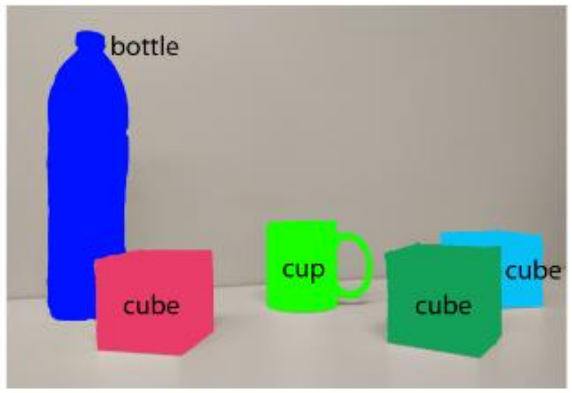
(a) Image classification



(b) Object localization



(c) Semantic segmentation



(d) Instance segmentation

左图展示了四种主要的计算机视觉任务（由粗粒度到精细粒度）

（a）、图像分类（what）

为图像赋予一个或多个语义标签

（b）、目标检测（ what & where ）

找到图像中物体的类别及所在位置

（c）、图像语义分割（ what & where ）

确定图像中物体的类别并精确勾勒出其所在位置

（d）、图像实例分割（ what & where ）

多个同类物体存在时，将它们一一区分出来

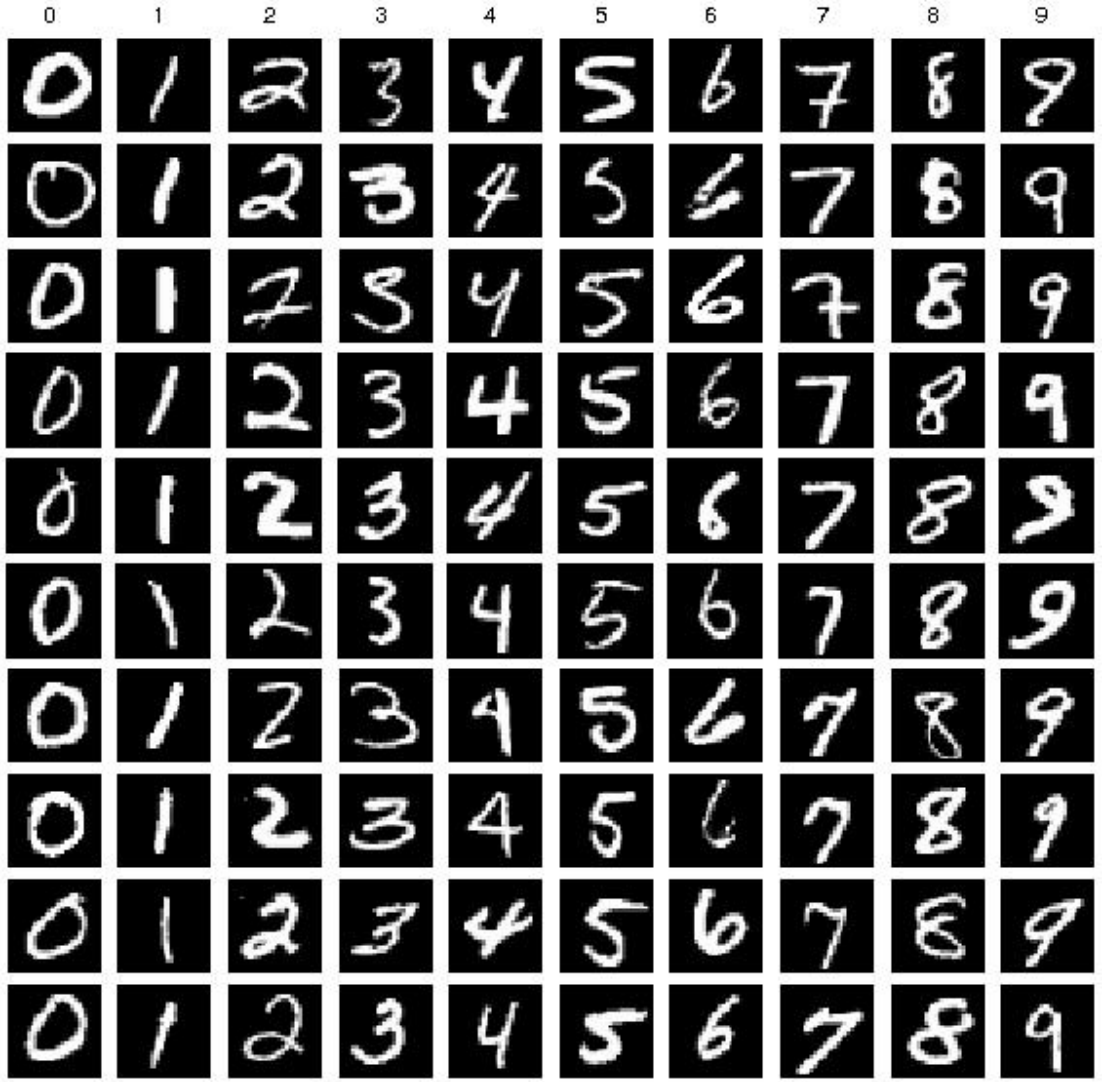
计算机视觉技术伴随着大规模数据集，得到快速发展和演进

图像分类：MNIST

https://yann.lecun.com/exdb/mnist/

- ✓ 70000张 28*28 的黑白图像
- ✓ 10个互斥的类别，分别为数字0-9
- ✓ 每个图片是黑底白字的形式，黑底用0表示，白字用0-1之间的浮点数表示，越接近1，颜色越白
- ✓ 训练：60000张

  每类图像随机抽取，张数不完全平均
- ✓ 测试：10000张



图像分类：CIFAR-10

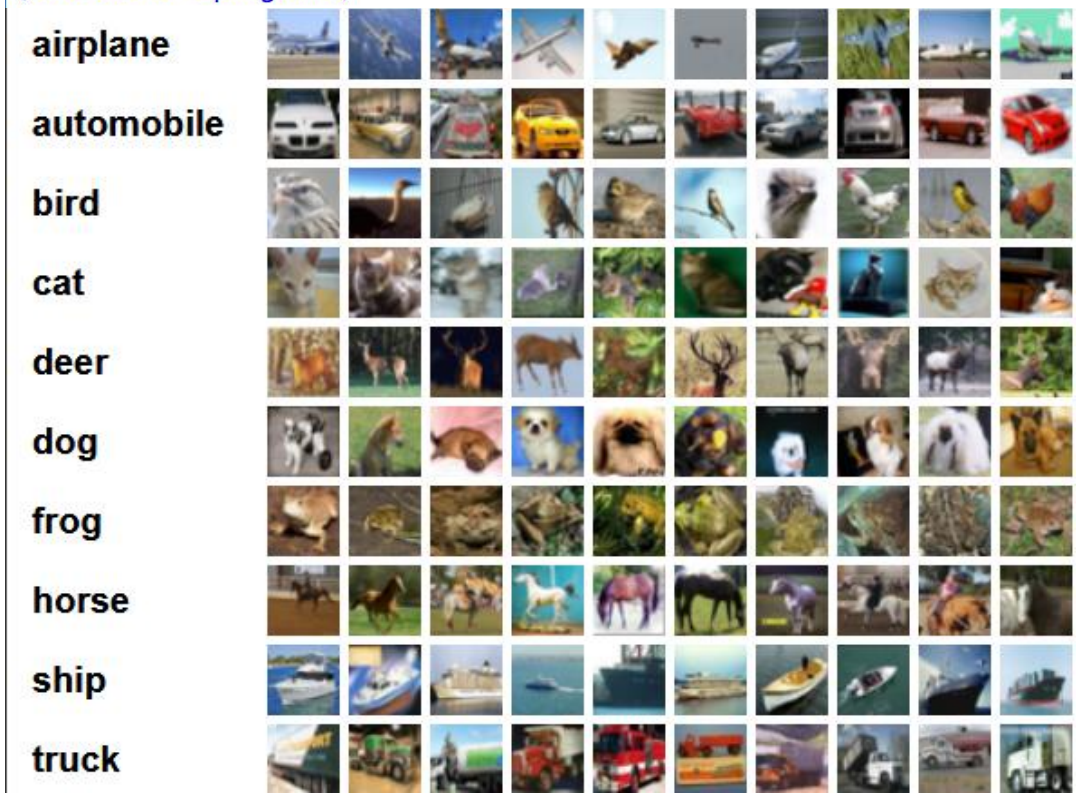
http://www.cs.toronto.edu/~kriz/cifar.html

- ✓ 60000张 32*32*3 的RGB图像
- ✓ 10个互斥的类别，每类6000张图片
- ✓ 训练：50000张，
  - 5个训练批，每批10000张
  - 每类图像随机抽取，张数不完全平均
- ✓ 测试：10000张，
  - 单独构成一批，每类1000张

Download

If you're going to use this dataset, please cite the tech report at the bottom of this page.

Version	Size	md5sum
CIFAR-10 python version	163 MB	c58f30108f718f92721af3b95e74349a
CIFAR-10 Matlab version	175 MB	70270af85842c9e89bb428ec9976c926
CIFAR-10 binary version (suitable for C programs)	162 MB	c32a1d4ab5d03f1284b67883e8d87530



图像分类：ImageNet

http://image-net.org/

1）Total number of non-empty synsets: **21841**

2）Total number of images: **14,197,122**

3）Number of images with bounding box annotations: **1,034,908**

4）Number of synsets with SIFT features: **1000**

5）Number of images with SIFT features: **1.2 million**



ImageNet大规模图像分类任务

训练集：1,281,167张图片+标签
验证集：50,000张图片+标签
测试集：100,000张图片

1000个类别，每个类别1000多张图像

✓ 可供用于图像分类、目标定位、目标检测、实例分割等多个计算机视觉任务。

✓ ImageNet国际计算机视觉挑战赛(ILSVRC)

卷积神经网络发展史

早期尝试

Hubel & Wiesel

LeNet

使用MNIST数据集，这是最早用于数字识别的CNN

历史突破

Dropout
ReLU

AlexNet

2012 ILSVRC远超第2名

ZF Net，基于AlexNet，2013 ILSVRC冠军

发展和演化

模型轻量化

Squeeze Net

Mobile Net

网络加深

VGG

2014 ILSVRC，图像识别略差于GoogLeNet，但是在很多图像分析问题(比如object detection)上效果好

增强卷积模块功能

NIN

GoogLeNet

Inception V3,V4

增加新的功能单元

Inception V2，BN

2014 ILSVRC冠军

跳跃连接

ResNet

2015年ILSVRC，Classification获得第一名

密集连接

DenseNet

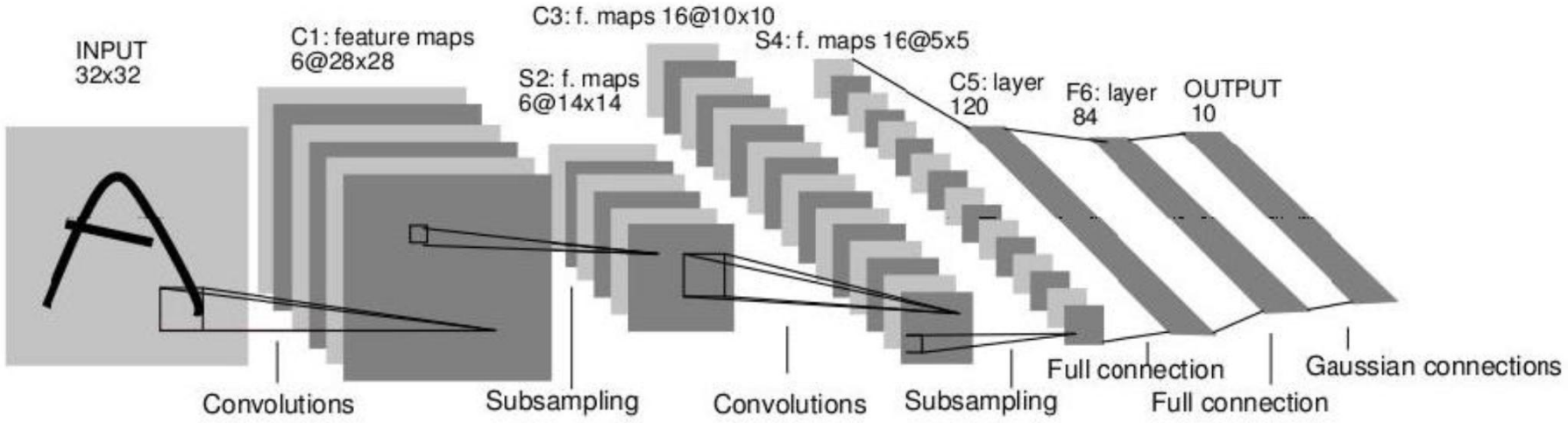
2017年ILSVRC

进一步降低Classification错误率

LeNet



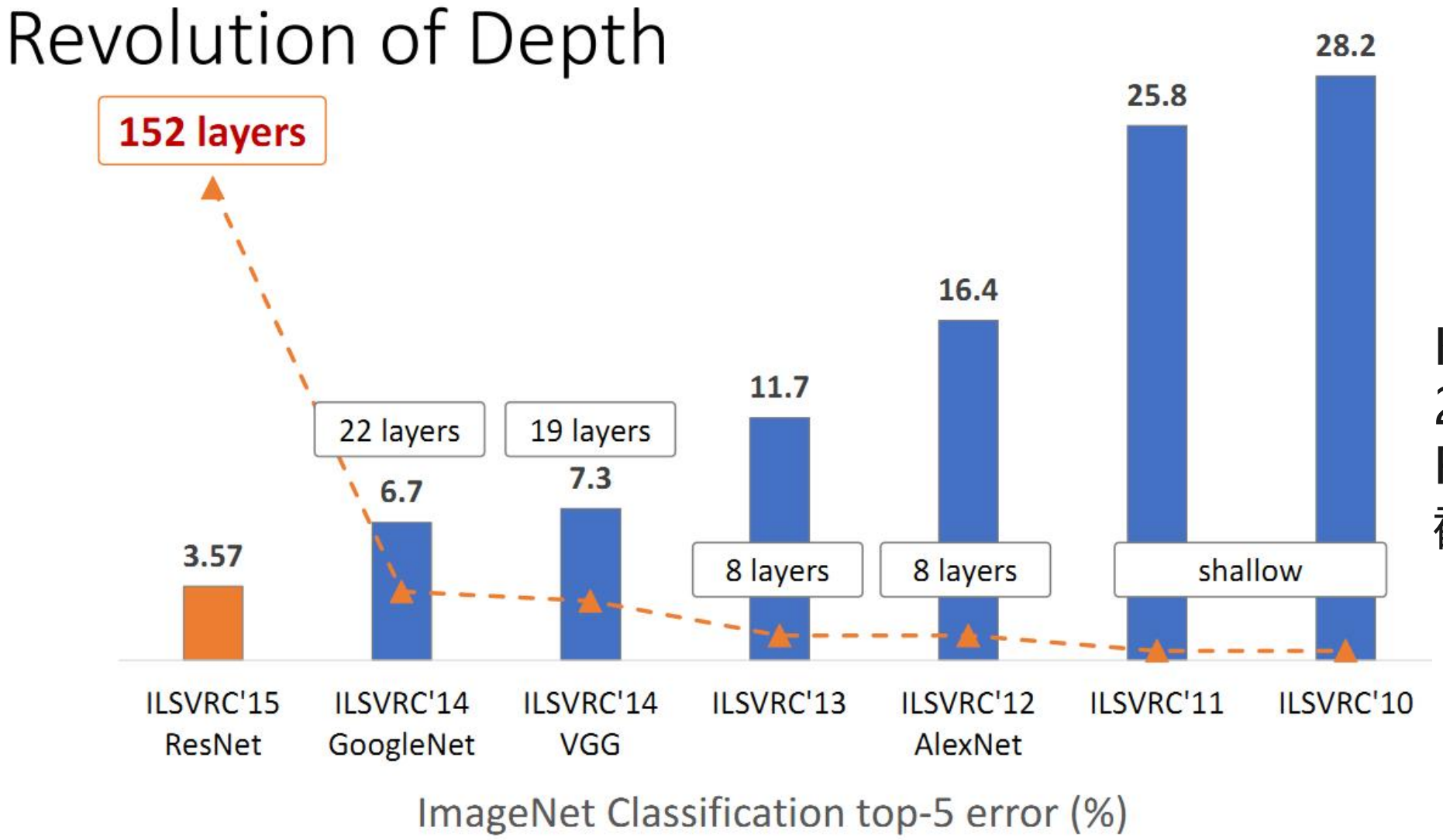
Yann LeCun
纽约大学教授
Meta AI Lab主任



Layer	Kernels	Stride	Padding	Output	Parameters
Input				32*32*1	
C1	5*5*1 filters * 6	1	Valid	28*28*6	(5*5*1+1)*6
S2	2*2 filter	2		14*14*6	
C3	5*5*6 filters * 16	1	Valid	10*10*16	(5*5*6+1)*16
S4	2*2 filter	2		5*5*16	
C5	5*5*16 filters * 120	1		120	(5*5*16)*120
F6				84	120*84
Output				10	84*10

Lecun Y, Bottou L. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11):2278-2324.

AlexNet

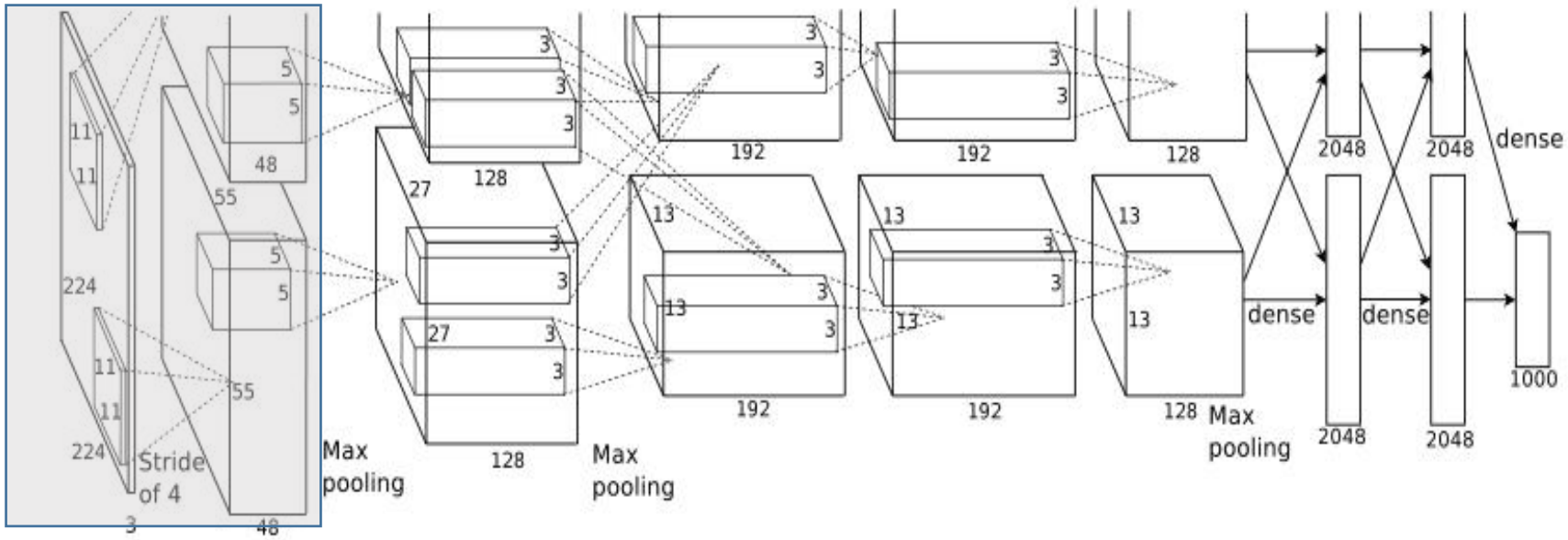


Kaiming He 2015年报告ResNet时的PPT截图

AlexNet

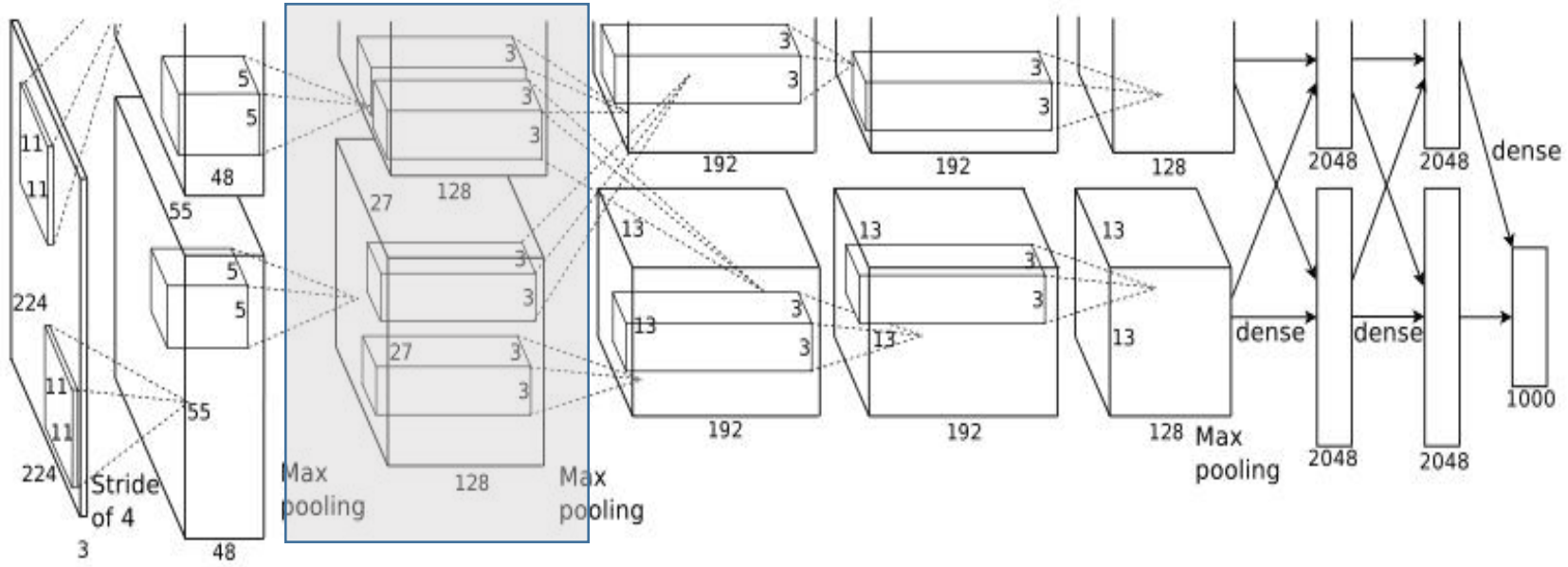
为什么画成上下两层?

卷积神经网络的里程杯：AlexNet



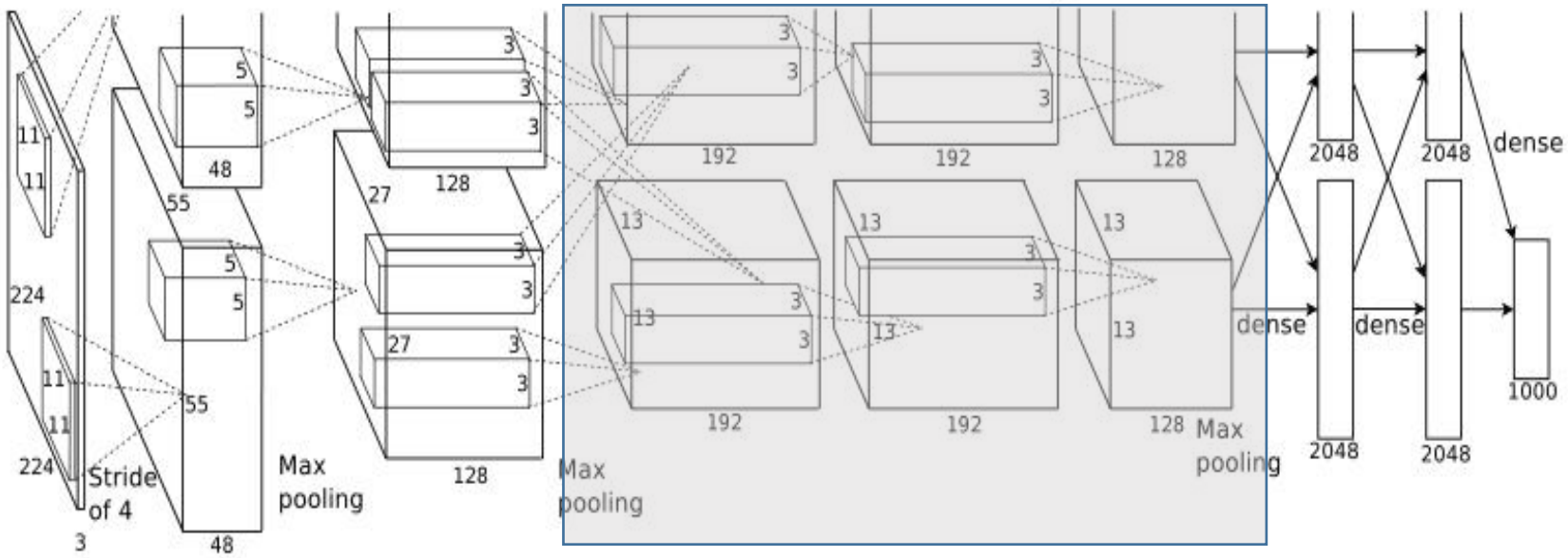
输入图像 （大小 227×227×3）			
卷积层 1 227×227×3	卷积核大小 11×11，数量 48 个，步长 4	卷积核大小 11×11，数量 48 个，步长 4	两台 GPU 同时训练。即共 96 个核。 输出特征图像大小：（227 - 11）/ 4 + 1 = 55，即 55×55×96
	激活函数（relu）	激活函数（relu）	
	池化（kernel size=3, stride=2）	池化（kernel size=3, stride=2）	输出特征图像大小：（ 55 - 3）/ 2 + 1 = 27，即 27×27×96
	标准化		

AlexNet



卷积层 2 27×27×96	卷积核大小 5×5，数量 128 个，步长 1	卷积核大小 5×5，数量 128 个，步长 1	输入特征图像先扩展 2 个像素，即大小 31×31 输出特征图像大小：（ 31 - 5）/ 1 + 1 = 27，即 27×27×256
	激活函数（relu）	激活函数（relu）	
	池化（kernel size=3, stride=2）	池化（kernel size=3, stride=2）	输出特征图像大小：（ 27 - 3）/ 2 + 1 = 13，即 13×13×256
	标准化		

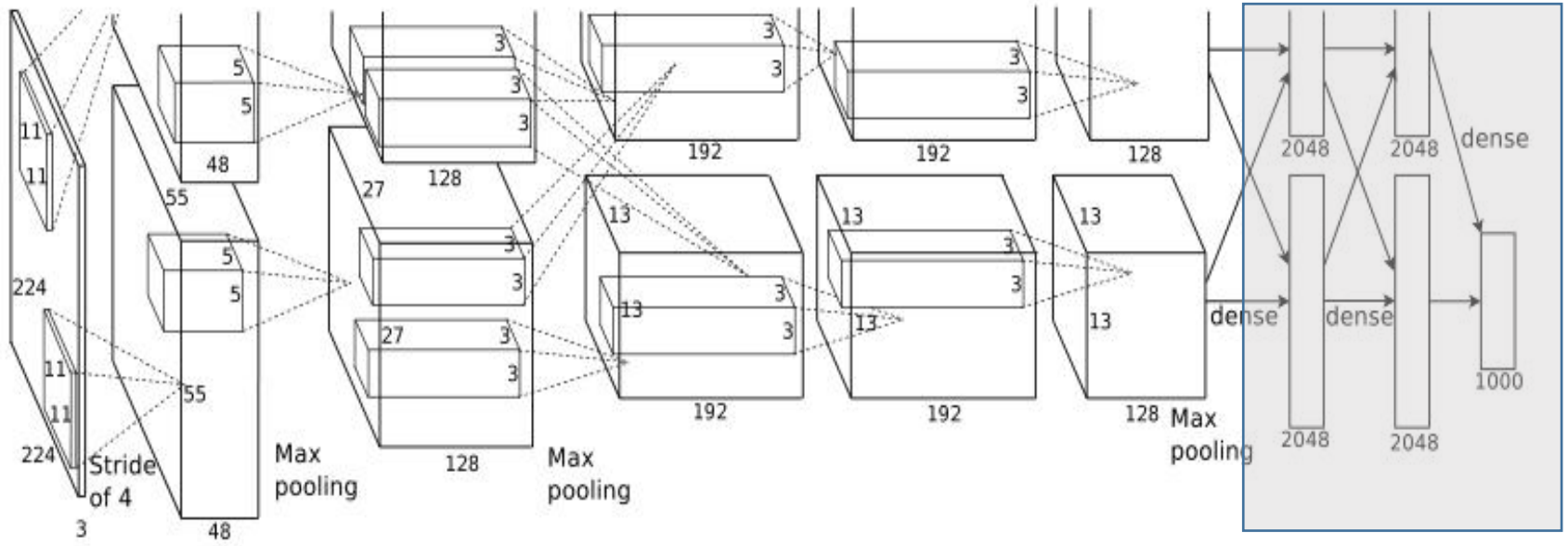
AlexNet



卷积层 3 13×13×256	卷积核大小 3×3，数量 192 个，步长 1	卷积核大小 3×3，数量 192 个，步长 1	输入特征图像先扩展 1 个像素，即大小 15×15 输出特征图像大小：（ 15 - 3）/1 + 1 = 13，即 13×13×384
	激活函数（relu）	激活函数（relu）	
卷积层 4 13×13×384	卷积核大小 3×3，数量 192 个，步长 1	卷积核大小 3×3，数量 192 个，步长 1	输入特征图像先扩展 1 个像素，即大小 15×15 输出特征图像大小：（ 15 - 3）/1 + 1 = 13，即 13×13×384
	激活函数（relu）	激活函数（relu）	
卷积层 5 13×13×384	卷积核大小 3×3，数量 128 个，步长 1	卷积核大小 5×5，数量 128 个，步长 1	输入特征图像先扩展 1 个像素，即大小 15×15 输出特征图像大小：（ 15 - 3）/ 1 + 1 = 13，即 13×13×256
	激活函数（relu）	激活函数（relu）	
	池化（kernel_size=3, stride=2）	池化（kernel_size=3, stride=2）	输出特征图像大小：（ 13 - 3）/ 2 + 1 = 6，即 6×6×256

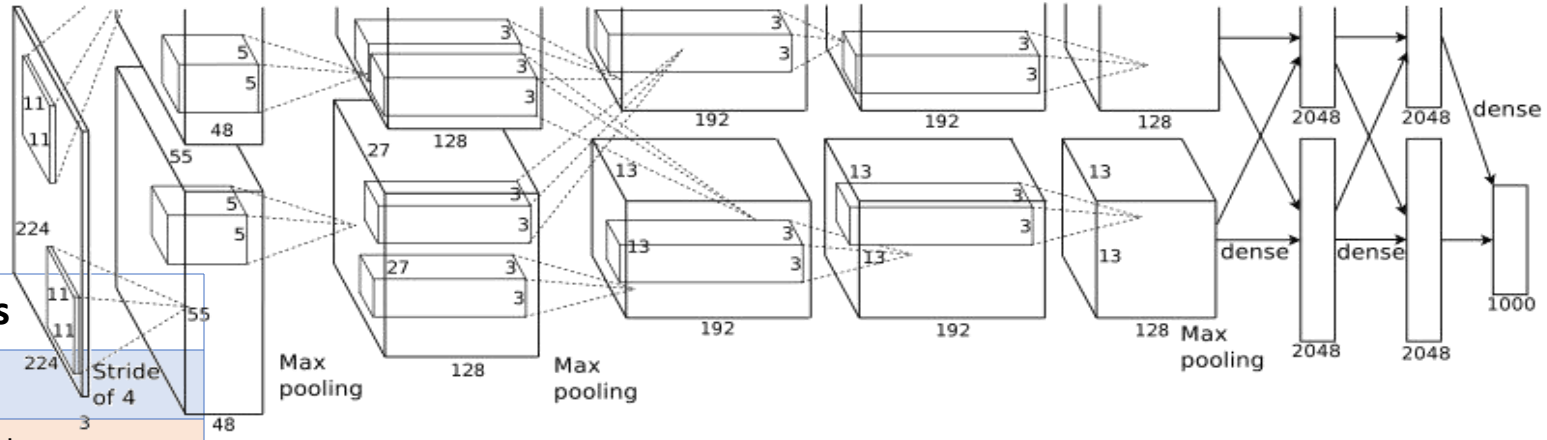
AlexNet

什么是好的创新?



全连接 6 6×6×256	2048 个神经元	2048 个神经元	共 4096 个神经元
	dropout	dropout	输出 4096×1 的向量
全连接 7 4096×1	2048 个神经元	2048 个神经元	共 4096 个神经元
	dropout	dropout	输出 4096×1 的向量
全连接 8 4096×1	1000 个神经元		输出 1000×1 的向量

AlexNet



Layer	Output	Parameters
Input	227*227*3	
Conv1	55*55*96	(11*11*3+1)*96=34944
MaxPool1	27*27*96	0
Norm1		
Conv2	27*27*256	(5*5*48+1)*128*2
MaxPool2	13*13*256	0
Norm2		
Conv3	13*13*384	(3*3*128*2+1)*384
Conv4	13*13*384	(3*3*192+1)*192*2
Conv5	13*13*256	(3*3*192+1)*128*2
MaxPool3	6*6*256	0
Fc6+Dropout	4096	(6*6*128*2+1)*4096
Fc7+Dropout	4096	(4096+1)*4096
Fc8	1000	(4096+1)*1000

✓ Details:

分组卷积

第一次使用ReLU

data augmentation

dropout = 0.5

batch-size = 128

SGD momentum = 0.9

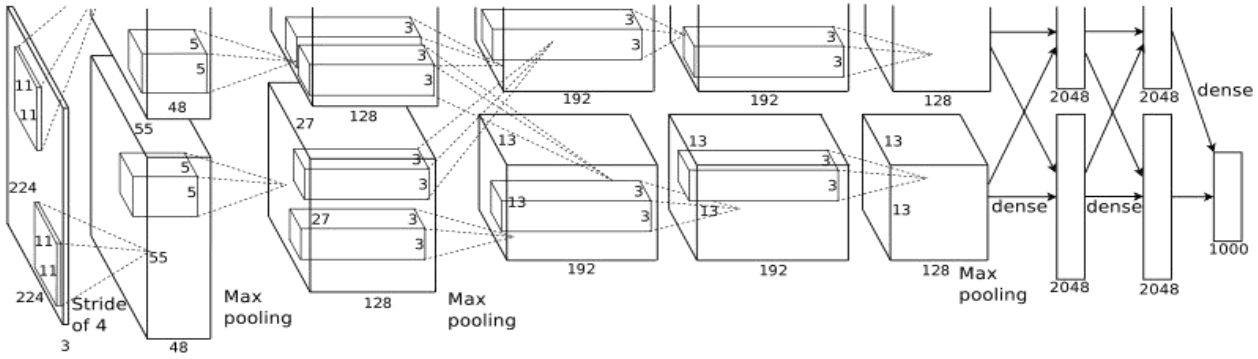
learning-rate = 1e-2

参数数量总数：60.9MB

卷积层参数：2.3MB，占总参数的3.8%

全连接层参数：58.6MB，占总参数量的96.2%

AlexNet



✓ Group Convolution（分组卷积）：

最早出现于AlexNet

硬件资源有限，卷积操作不能放在同一个GPU进行处理

Intuition：把feature maps分给多个GPU，对多个GPU的结果进行融合

✓ Group Convolution的计算量评估：

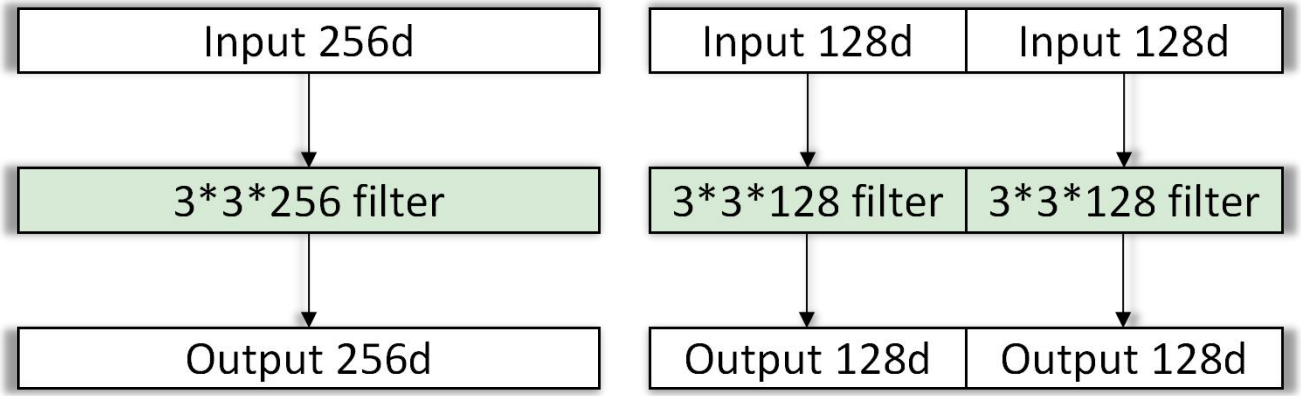
Input channel = 256，Output channel = 256，kernel = 3*3

不分组的卷积：256*3*3*256 parameters

分为2组的卷积：128*3*3*128 parameters * 2

每个GPU的计算量会降低到整个模型的1/groups^2

整体GPU的计算量会下降为1/groups

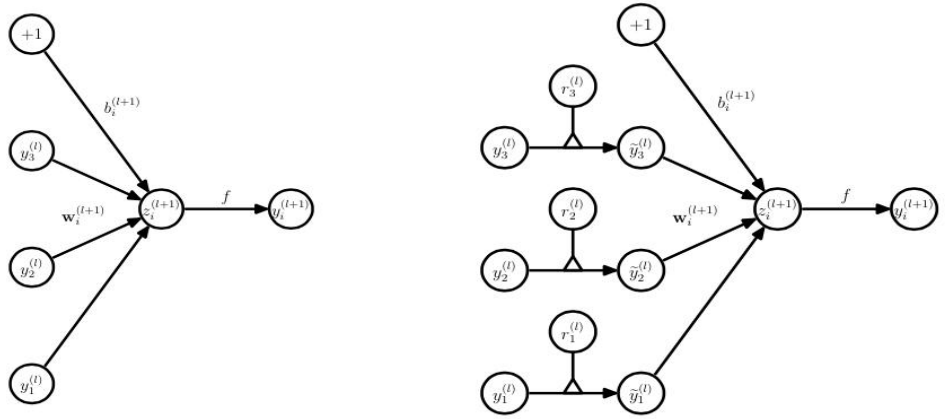


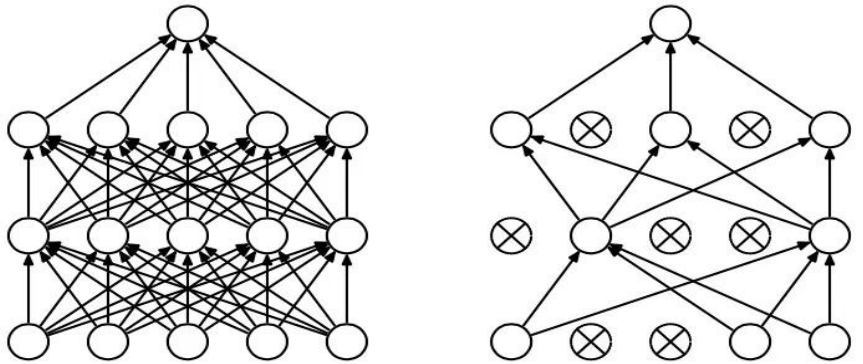
AlexNet

✓ Dropout：

1 引入概率的思想，训练时节点是否存在于网络中是由随机概率决定的

2 训练时加入随机性，进行反向传播算法。测试时，只是用训练好的参数

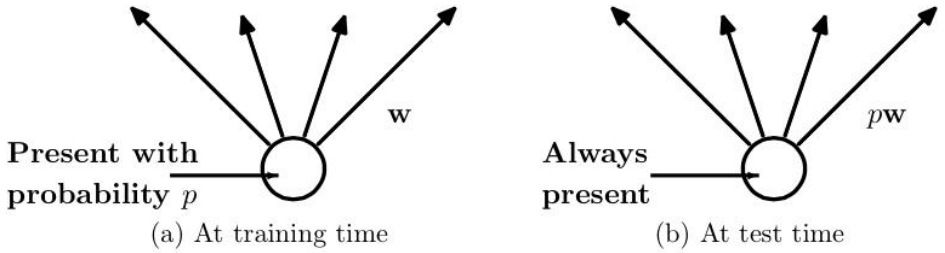




N个节点的网络，Dropout = 同时训练2^n个网络

是ensemble在分类性能上的一个近似，减弱了神经元节点间的联合适应性，在不大幅增加计算量的情形下防止过拟合，增强了泛化能力

dropout = 0.5是常用值，随机生成的网络结构容量适中



AlexNet

- ✓ Data Augmentation：

  拟合自然界中常见的噪声

  主要用于数据集较小的时候，可以丰富图像

  训练集、防止过拟合

- ✓ 常用的数据增强方式有：

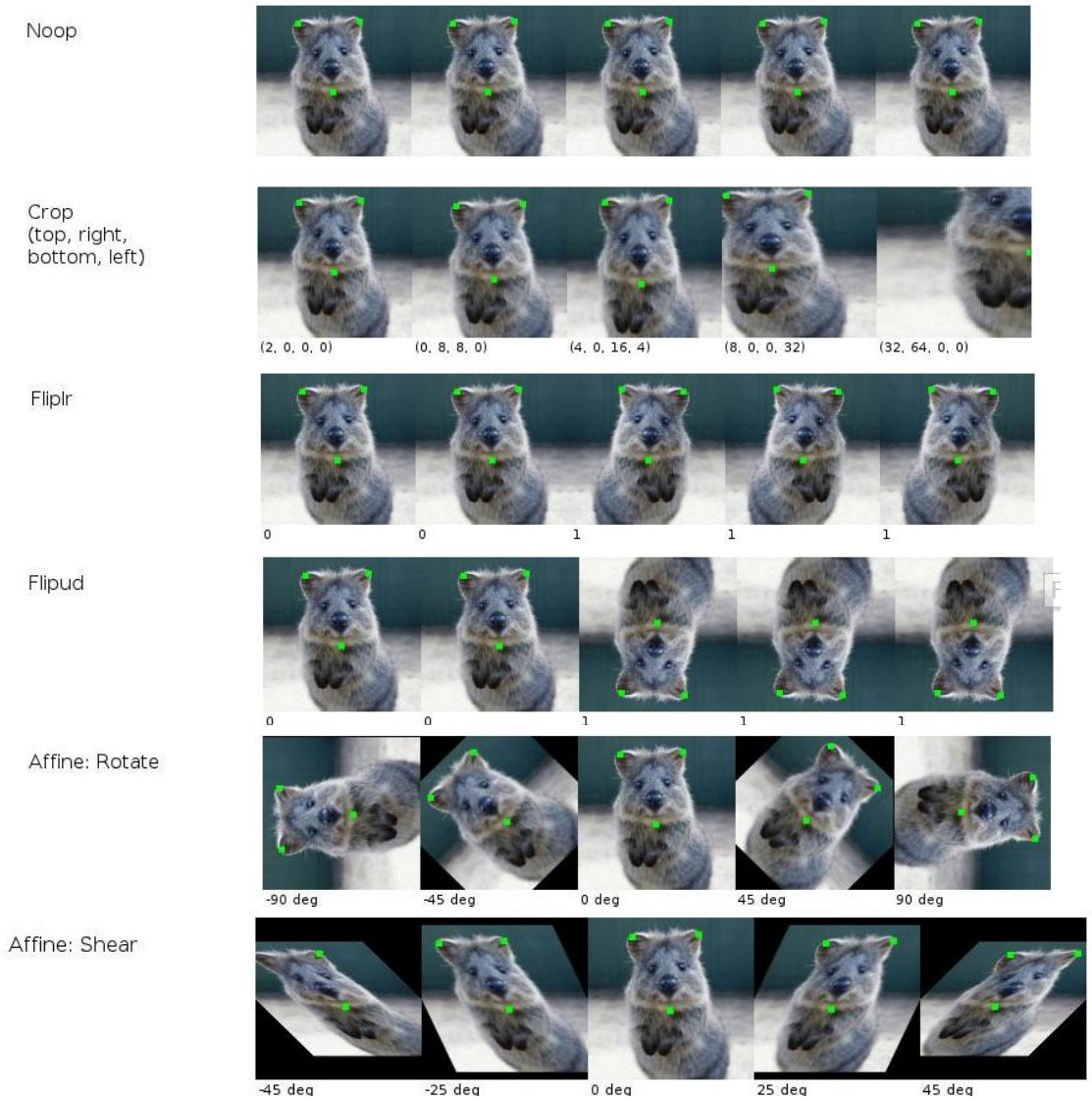
  对颜色的数据增强

  色彩的饱和度、亮度、对比度

  加噪声（高斯噪声）

  水平翻转、垂直翻转

  随机旋转、随机裁剪（crop）



AlexNet

Hinton谈AlexNet，靠谱的学生及运气



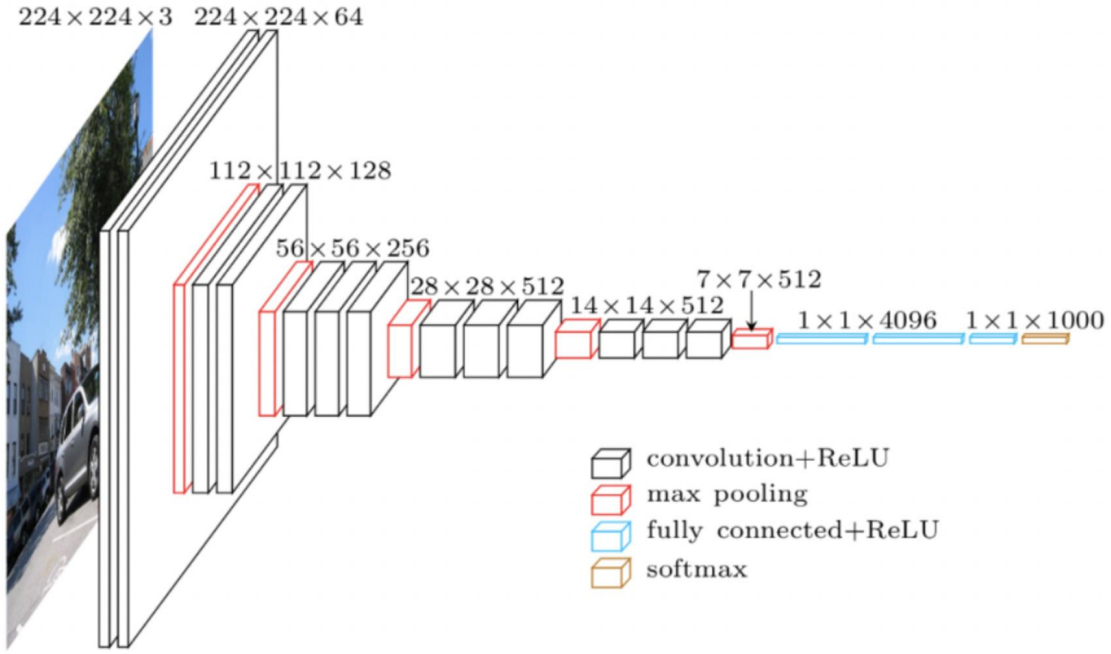
VGG

牛津大学视觉几何计算组

深度增加 & 小卷积核 → 对网络最后的分类识别效果有很大作用

小卷积核： 3*3：表示上下、左右、中心这些概念的最小卷积核尺寸

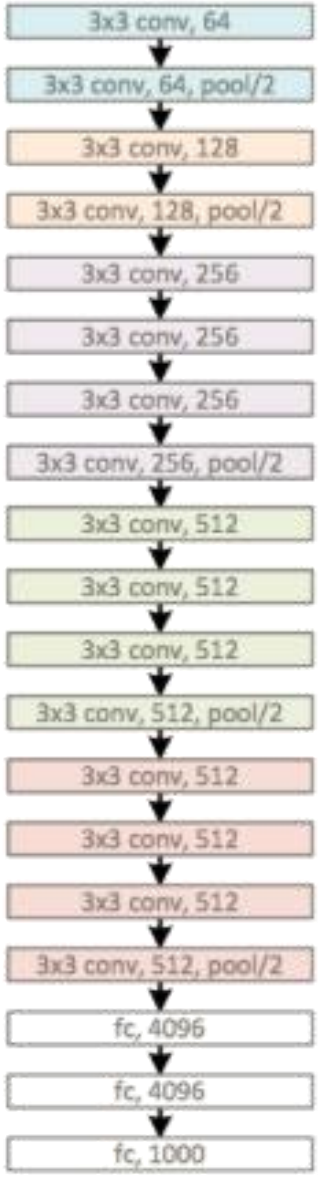
深度：AlexNet 8 层 → VGG最深19层



ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGG

- ✓ Details:
  - 3*3 conv-filter
  - conv-stride = 1
  - pad = 1, Same
  - 2*2 pool-filter
  - pool-stride = 2



VGG-19

Layer	Output	Parameters
Input	224*224*3	Note:未计算偏置
Conv3-64	224*224*64	(3*3*3)*64 = 1728
Conv3-64	224*224*64	(3*3*64)*64 = 36864
Pool2	112*112*64	0
Conv3-128	112*112*128	(3*3*64)*128 = 73728
Conv3-128	112*112*128	(3*3*128)*128 = 147456
Pool2	56*56*128	0
Conv3-256	56*56*256	(3*3*128)*256 = 294912
Conv3-256	56*56*256	(3*3*256)*256 = 589824
Conv3-256	56*56*256	(3*3*256)*256 = 589824
Pool2	28*28*256	0
Conv3-512	28*28*512	(3*3*256)*512 = 1179648
Conv3-512	28*28*512	(3*3*512)*512 = 2359296
Conv3-512	28*28*512	(3*3*512)*512 = 2359296
Pool2	14*14*512	0
Conv3-512	14*14*512	(3*3*512)*512 = 2359296
Conv3-512	14*14*512	(3*3*512)*512 = 2359296
Conv3-512	14*14*512	(3*3*512)*512 = 2359296
Pool2	7*7*512	0
Fc	1*1*4096	(7*7*512)*4096
Fc	1*1*4096	4096*4096
Fc	1*1*1000	4096*1000

VGG

VGG的勇气值得学习！

✓ Small Convolution filter：

AlexNet用到一些非常大的卷积核，比如11*11、5*5。

Intuition：感受野越大、看到的图片信息就越多，获得的特征会越好

参数和计算量的增加，如何衡量该使用多大的卷积核？

Methods：使用2个3*3的卷积核的组合比用1个5*5的卷积核

效果更佳 && 参数量降低

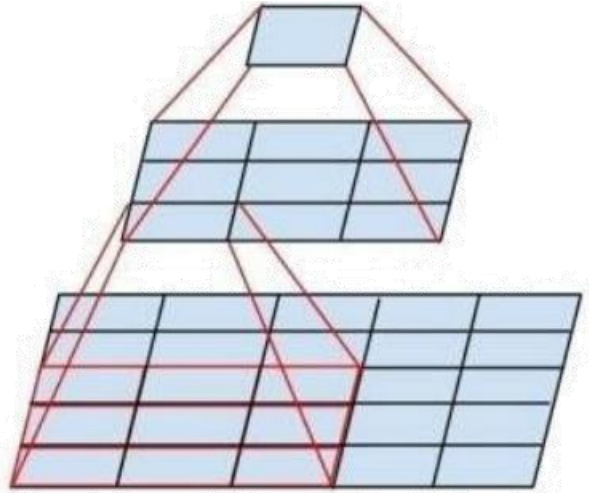


Figure 1. Mini-network replacing the 5×5 convolutions.

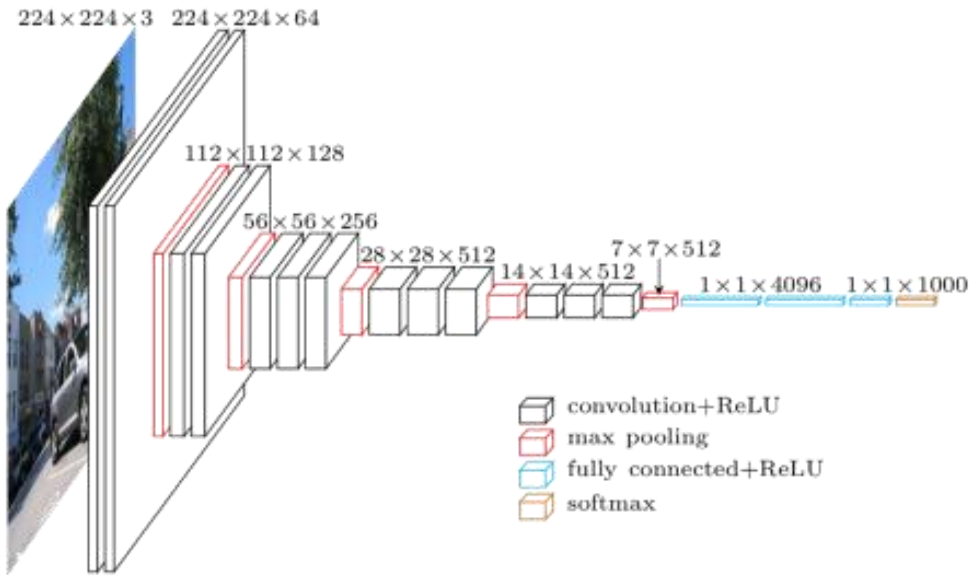
✓ 参数量的评估：

以256通道的隐层数据为例：

2个3*3的卷积核	==	1个5*5的卷积核
2*3*3*256 params		5*5*256 params
3个3*3的卷积核	==	1个7*7的卷积核
3*3*3*256 params		7*7*256 params

VGG

VGG-16的结构和参数量




```
INPUT: [224x224x3]        memory:  224*224*3=150K   weights: 0
CONV3-64: [224x224x64]  memory:  224*224*64=3.2M   weights: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64]  memory:  224*224*64=3.2M   weights: (3*3*64)*64 = 36,864
POOL2: [112x112x64]  memory:  112*112*64=800K   weights: 0
CONV3-128: [112x112x128]  memory:  112*112*128=1.6M   weights: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128]  memory:  112*112*128=1.6M   weights: (3*3*128)*128 = 147,456
POOL2: [56x56x128]  memory:  56*56*128=400K   weights: 0
CONV3-256: [56x56x256]  memory:  56*56*256=800K   weights: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256]  memory:  56*56*256=800K   weights: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256]  memory:  56*56*256=800K   weights: (3*3*256)*256 = 589,824
POOL2: [28x28x256]  memory:  28*28*256=200K   weights: 0
CONV3-512: [28x28x512]  memory:  28*28*512=400K   weights: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512]  memory:  28*28*512=400K   weights: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512]  memory:  28*28*512=400K   weights: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512]  memory:  14*14*512=100K   weights: 0
CONV3-512: [14x14x512]  memory:  14*14*512=100K   weights: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512]  memory:  14*14*512=100K   weights: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512]  memory:  14*14*512=100K   weights: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512]  memory:  7*7*512=25K  weights: 0
FC: [1x1x4096]  memory:  4096  weights: 7*7*512*4096 = 102,760,448
FC: [1x1x4096]  memory:  4096  weights: 4096*4096 = 16,777,216
FC: [1x1x1000]  memory:  1000 weights: 4096*1000 = 4,096,000

TOTAL memory: 24M * 4 bytes ~= 93MB / image (only forward! ~*2 for bwd)
TOTAL params: 138M parameters
```

卷积层与全连接层
全连接层之间

```
conv3-64  x 2         : 38,720
conv3-128 x 2         : 221,440
conv3-256 x 3         : 1,475,328
conv3-512 x 3         : 5,899,776
conv3-512 x 3         : 7,079,424
fc1                   : 102,764,544
fc2                   : 16,781,312
fc3                   : 4,097,000
TOTAL                 : 138,357,544
```

内存消耗（部分）：

结构部分：93MB

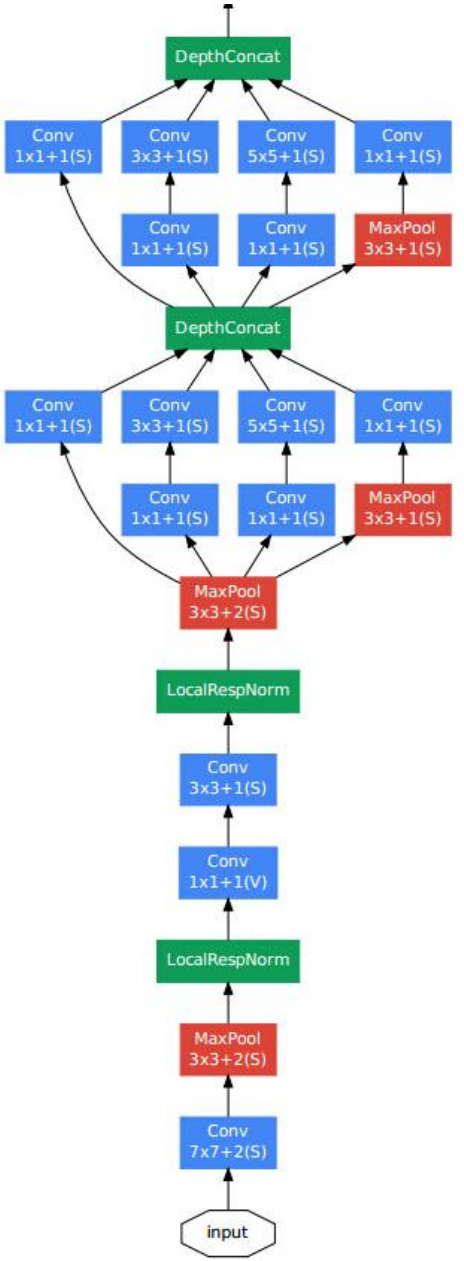
参数部分：138MB * 4B = 526MB

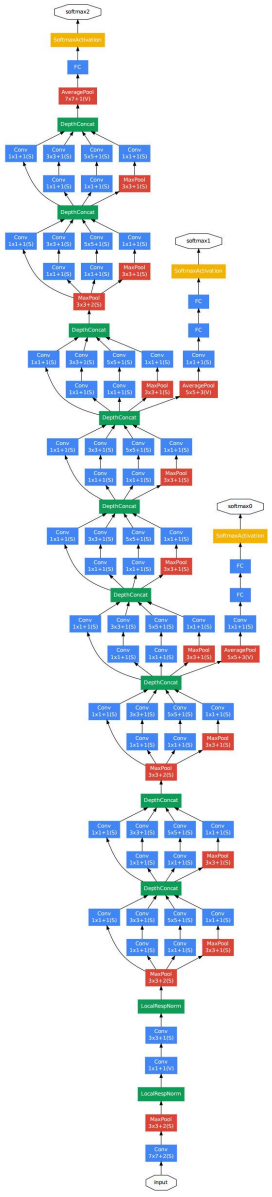
训练过程消耗更多

GoogLeNet/Inception

Google提出的神经网络，一共有V1-V4共4个不同的模型

- ✓ **使用了Inception模块**
  - 可以并行执行多个具有不同尺度的卷积运算或池化操作
  - 将多个卷积核卷积的结果拼接成一个更深的特征图
- ✓ **使用了大量的trick提高网络性能**
  - Bottleneck（瓶颈）：1*1的卷积核，借鉴NIN
  - 使用全局平均池化GAP连接卷积层和全连接层
  - 在v2中，采用Batch Normalization（批归一化）
  - 在v3中，采用非对称卷积降低运算量
  - 在v4中，结合了ResNet中的思想，发现Residual Connections貌似只能加快网络收敛速度，是更大的网络规模提高了精度





Szegedy C , Liu W , Jia Y , et al. Going Deeper with Convolutions, CVPR 2015.

GoogLeNet/Inception

✓ Multi-size filters in one layer（Inception Block）：

最早出现于Inception-v1/GoogleNet

传统的堆叠式网络，每层仅用一个尺寸的卷积核。

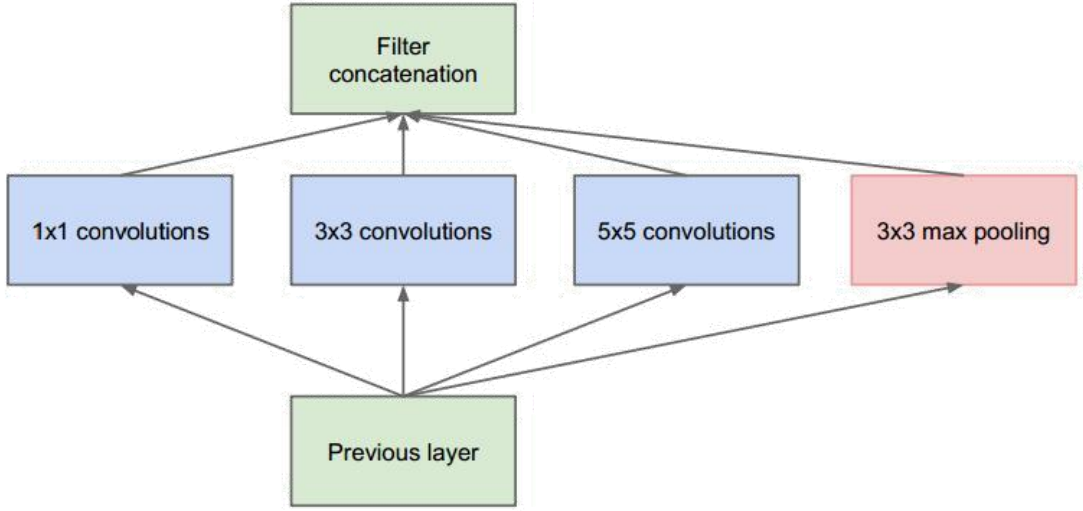
VGG每层只使用3*3的卷积核。

Intuition：信息尺寸的差异，使得不同尺度的特征结合起来可以得到更好的特征表示

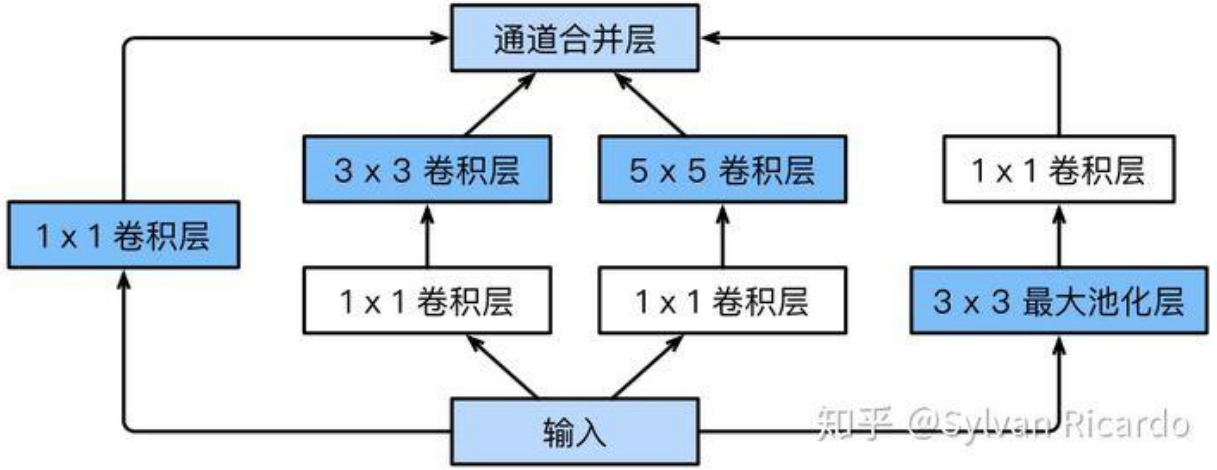
Methods：卷积核设为1，3，5，stride设为1

可以使用 pad = 0, 1, 2 很方便的对齐

结合方式：在depth维度上Concatenation（拼接）



(a) Inception module, naïve version



GoogLeNet/Inception

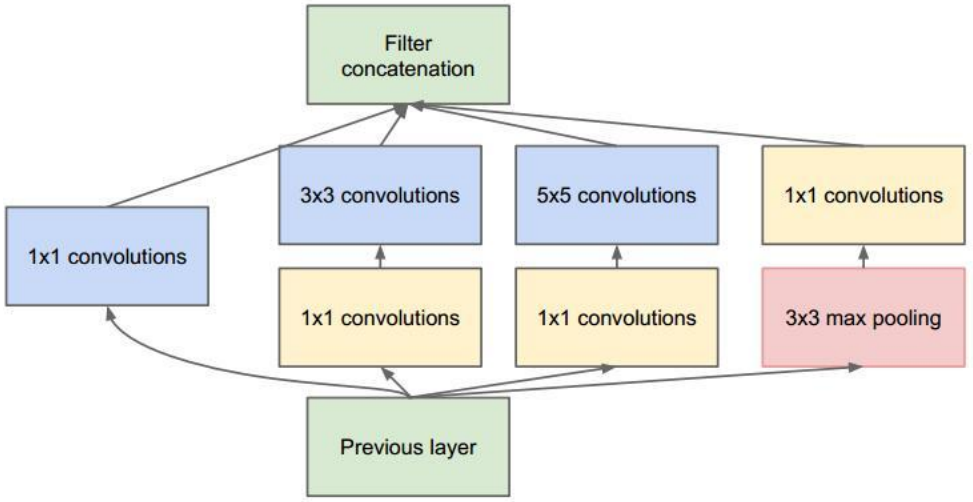
✓ Bottleneck：

最早出现于Network in Network

在同一层使用多个不同尺寸的卷积核会导致参数过多

Intuition：Network in Network中1*1的卷积核的启发

在原来的Inception结构中加入1*1的卷积核



(b) Inception module with dimension reductions

✓ Bottleneck参数量评估：

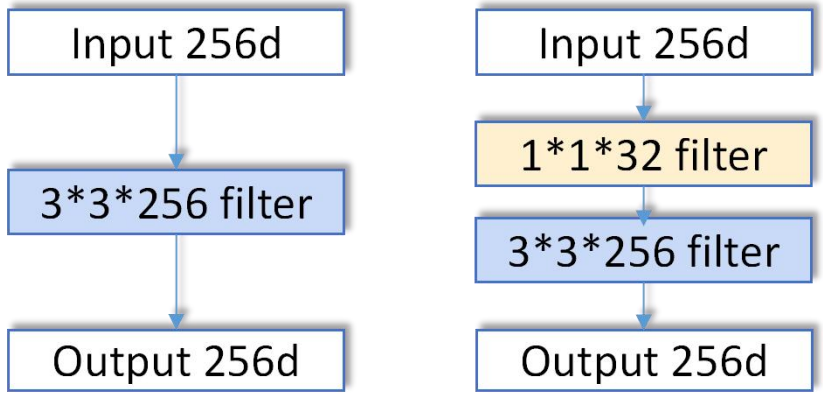
Input channel = 256，Output channel = 256

卷积核 = 3*3*256

参数量：256*3*3*256 = 589,824

卷积核：1*1*32，3*3*256

参数量：256*1*1*32+32*3*3*256=81,920



GoogLeNet/Inception

✓ 使用全局平均池化GAP代替全连接：

多设计几个全连接层可以提升神经网络的分类性能，

全连接层一度成为标配

Intuition：全连接层参数量巨大，

特别是与最后一个卷积层相连接的全连接层

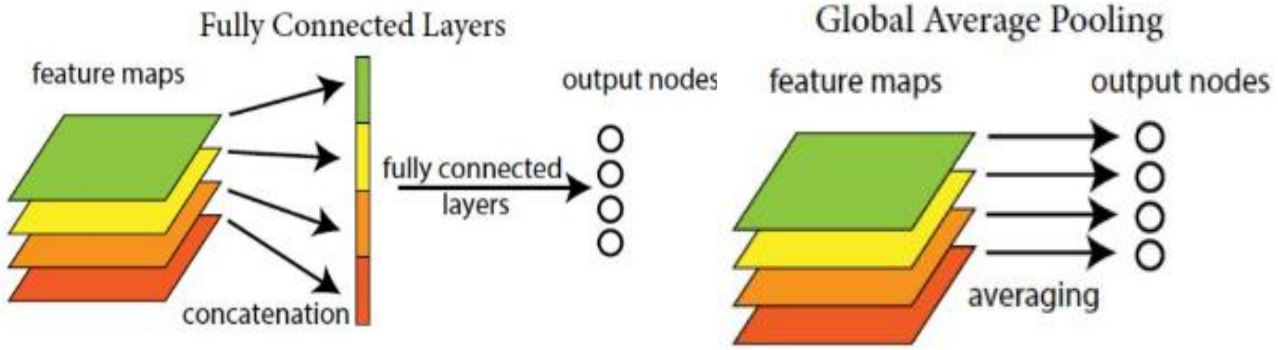
Methods：全局平均池化 Global Average Pooling

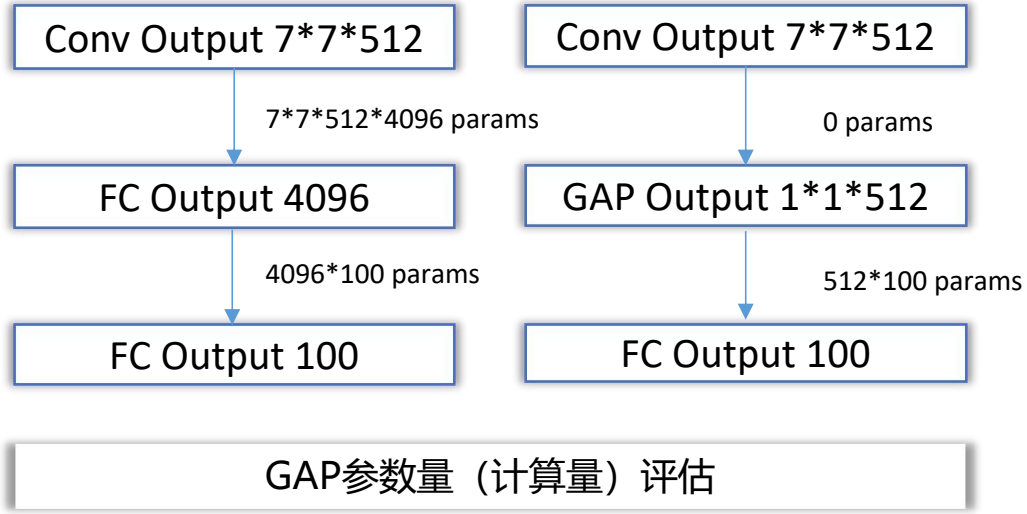
代替卷积与全连接的连接

Advantage：参数量极大降低，可以实现任意图像大小的输入

InceptionV3 参数量：23.6MB，

推理内存消耗：35.5MB





GoogLeNet/Inception

✓ Batch Normalization（批归一化）：

最早出现于Inception v2

训练的过程中，隐层输入的分布一直在变化

学习的过程又要使每一层适应输入的分布，

会导致反向传播时深层神经网络训练的局部差异和不稳定

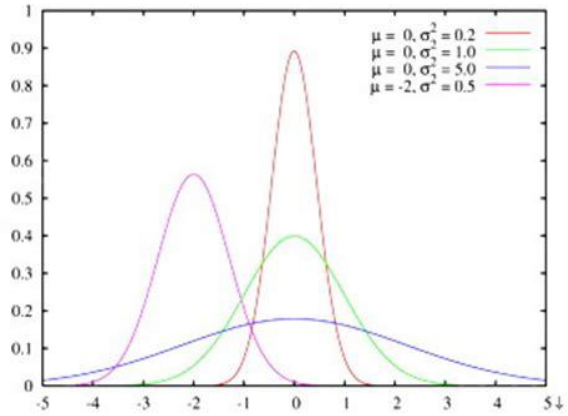
这是训练深层神经网络收敛越来越慢的**本质原因**

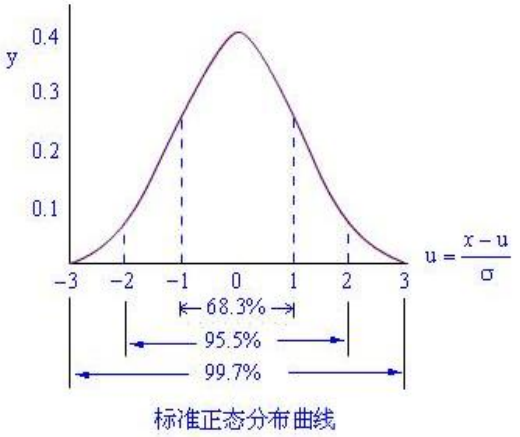
Intuition：在训练过程中强行将每一层的输入

拉回到比较标准的正态分布

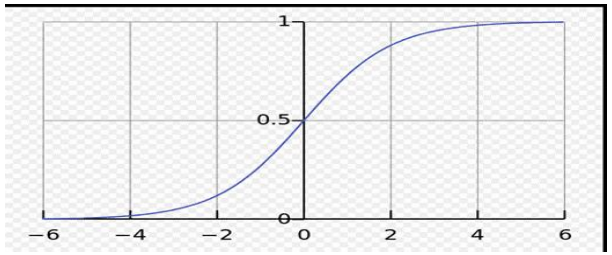
可以防止梯度消失或爆炸

可以加快训练速度





标准正态分布曲线



ResNet

微软提出的神经网络，CVPR当年的最佳论文

✓ **使用了恒等映射**

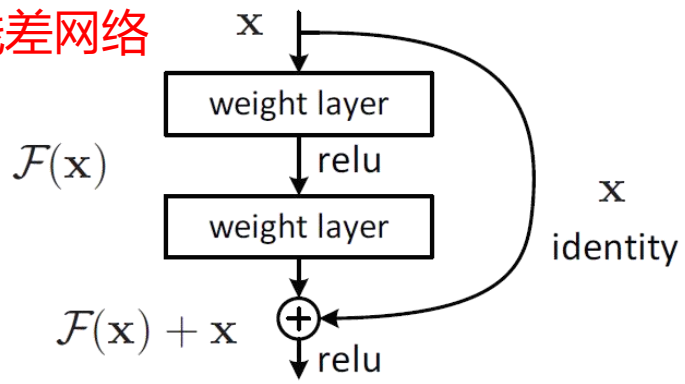
传统神经网络训练的函数为F(x)

添加恒等映射后，神经网络训练的函数变为F(x)+x

作者认为这样训练出来的网络，相当于是在对x作修正，

修正的幅度就是F(x)， F(x)在数学上称为残差

所以作者提出的网络称为残差网络



method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43†
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	**19.38**	**4.49**

ResNets @ ILSVRC & COCO 2015 Competitions

- **1st places in all five main tracks**
  - ImageNet Classification: "*Ultra-deep*" 152-layer nets
  - ImageNet Detection: 16% better than 2nd
  - ImageNet Localization: 27% better than 2nd
  - COCO Detection: 11% better than 2nd
  - COCO Segmentation: 12% better than 2nd

He K , Zhang X , Ren S , et al. Deep Residual Learning for Image Recognition, CVPR 2016.

ResNet

- ✓ 两个相同深度网络的比较（训练误差方面）

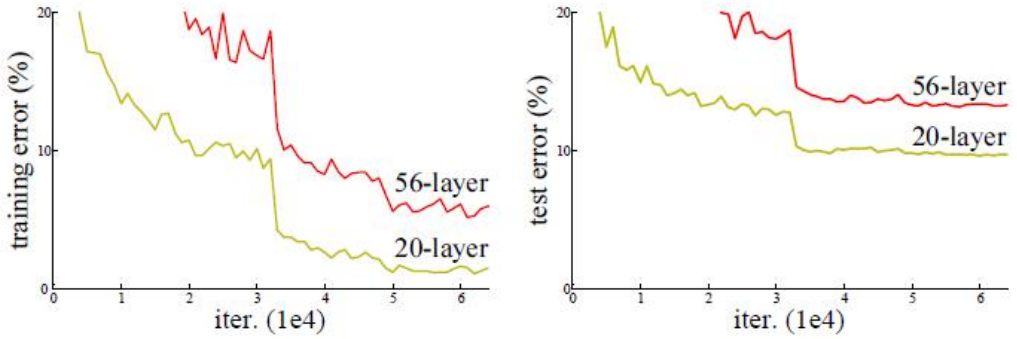
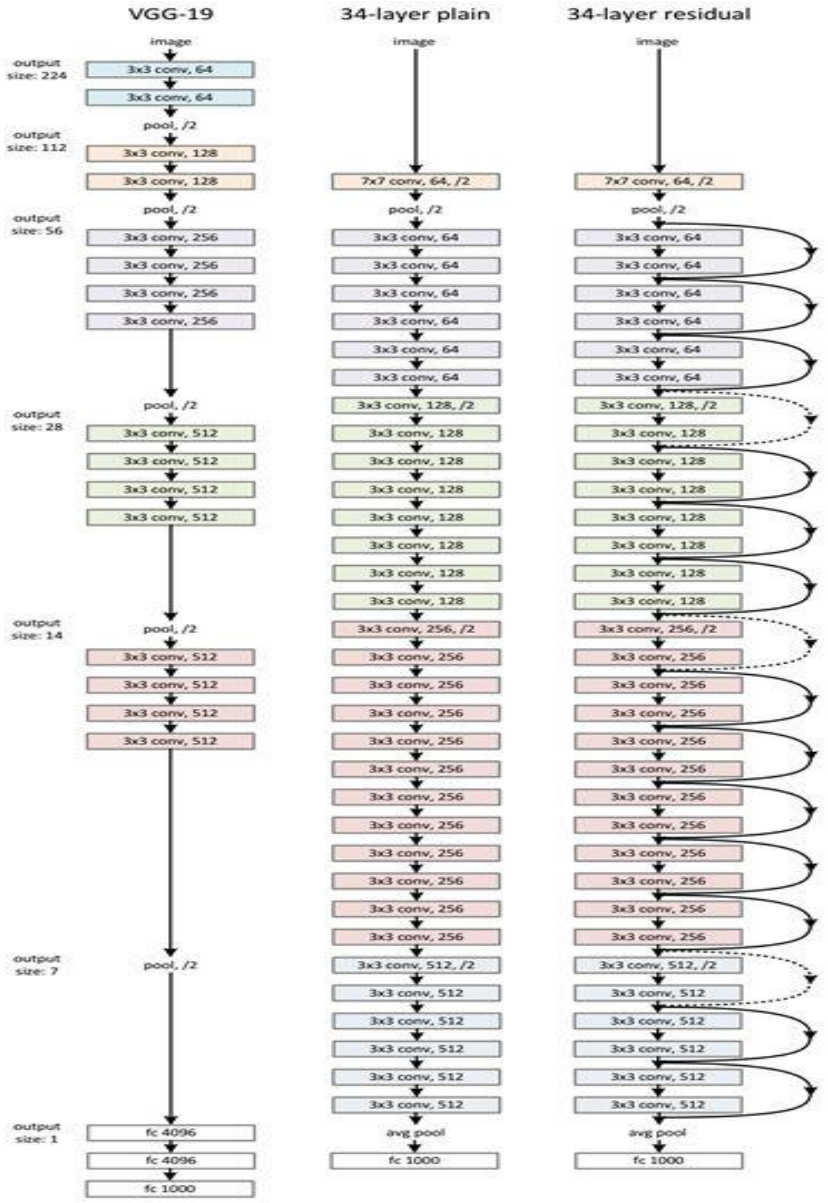


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.



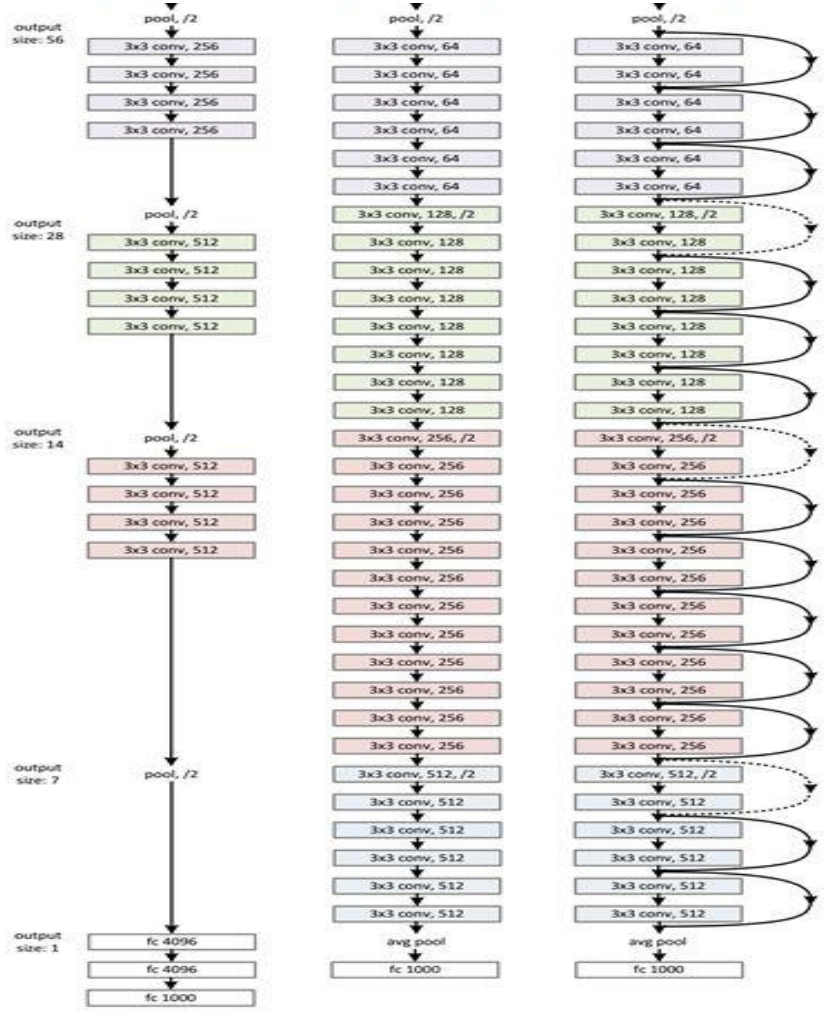
ResNet

Deep residual learning for image recognition

K He, X Zhang, S Ren, J Sun - … and **pattern recognition**, 2016 - openaccess.thecvf.com

… **Deeper** neural **networks** are more difficult to train. We present a **residual learning** framework to ease the training of **networks** that are substantially **deeper** than those used previously. …

☆ Save 99 Cite Cited by 235347 Related articles All 65 versions »



平凡网络中，为什么深度增加到一定程度后，训练的效果反而下降了?

梯度下降算法本身的缺陷：链式法则

深度增加后，相比浅一些的网络来说梯度减小。误差传播过程变慢，网络的优化速度就慢

残差网络中，为什么能够避免这种现象?

恒等映射下，虽然网络的深度加深，但是每层中都会有足够多的由梯度承载的信息量，梯度不会太小

提升了深层网络的训练稳定性

Transformer深至1000层还能稳定训练（DeepNorm），微软实习生一作（致敬ResNet)

https://baijiahao.baidu.com/s?id=1726252747886226968&wfr=spider&for=pc

Part 2

卷积神经网络发展概述（after ResNet)

DenseNet

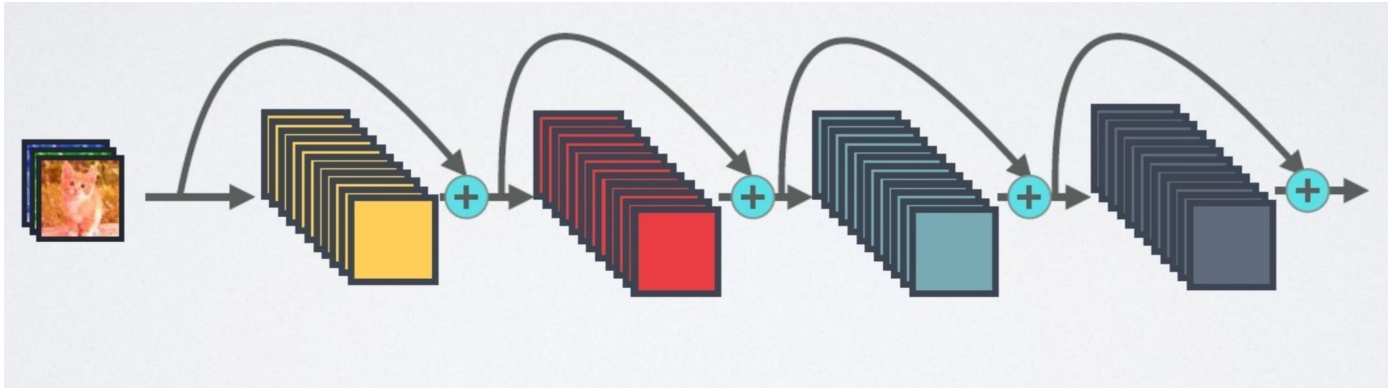
增加深度 V.S. 丰富连接

- 更激进的密集连接机制

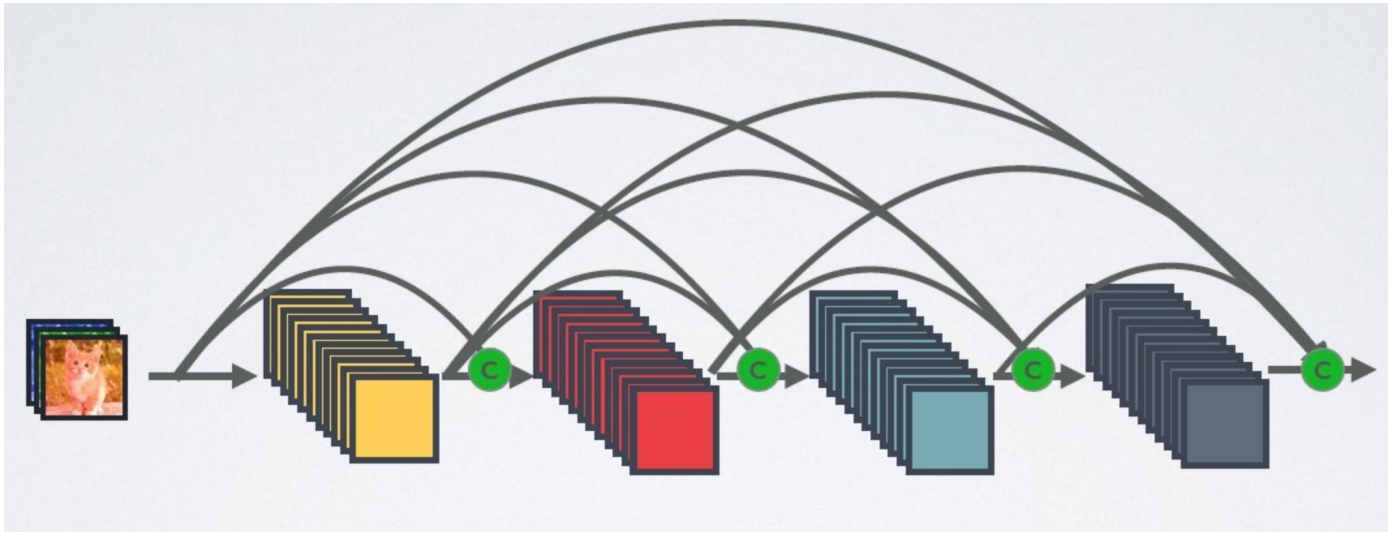
每个层都会接受其前面所有层作为其额外的输入

ResNet：元素级相加

DenseNet：channel级连接



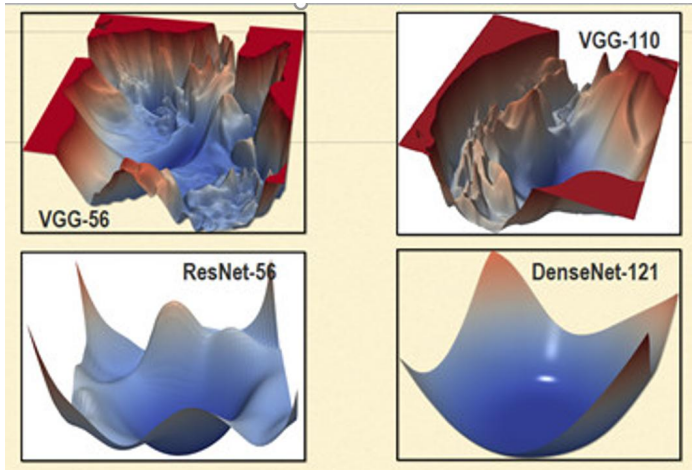
ResNet



DenseNet

神经网络损失函数可视化曲面：从VGG到DenseNet

越来越光滑，算法更容易找到好的优化点



Huang G , Liu Z , Laurens V D M , et al. Densely Connected Convolutional Networks, CVPR 2017.

ResNeXt

基于ResNet和GoogLeNet的改进

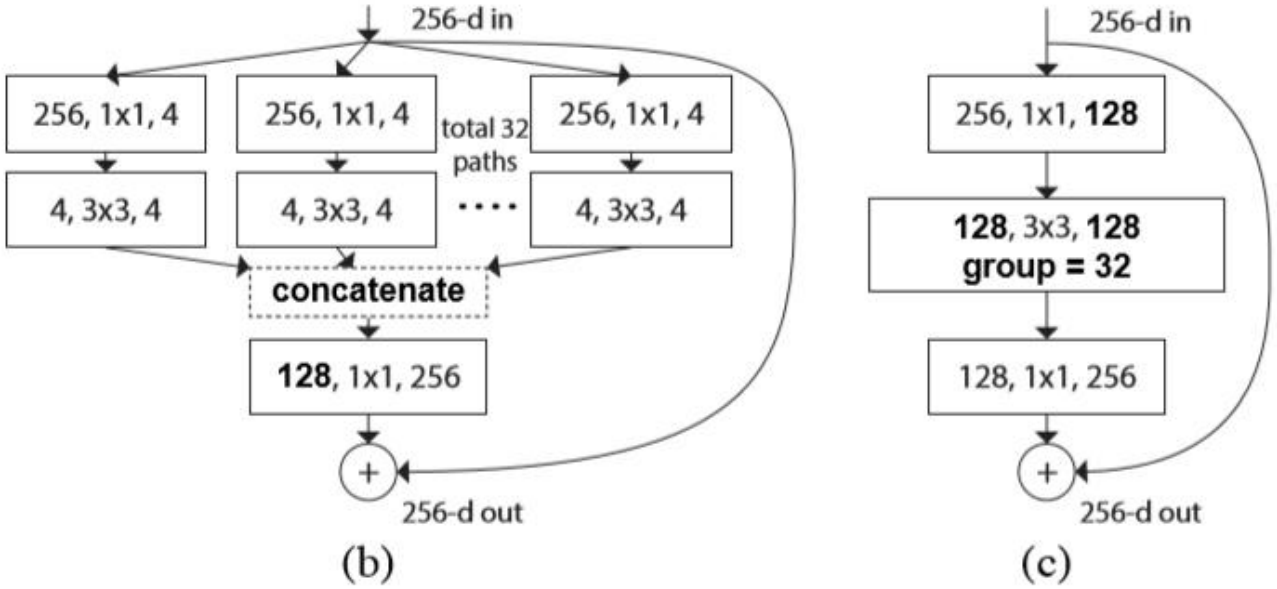
✓ **引入了基数的概念**

基数指变换的数量，即将网络划分成多少个子网络

将输入划分成多个低维嵌入，然后对每个低维嵌入使用相同的拓扑结构进行变换，最后将变换后的结果聚合在一起

增加基数比增加网络的深度或宽度更有效!

	224×224		320×320 / 299×299	
	top-1 err	top-5 err	top-1 err	top-5 err
ResNet-101 [14]	22.0	6.0	-	-
ResNet-200 [15]	21.7	5.8	20.1	4.8
Inception-v3 [39]	-	-	21.2	5.6
Inception-v4 [37]	-	-	20.0	5.0
Inception-ResNet-v2 [37]	-	-	19.9	4.9
ResNeXt-101 (**64** × **4d**)	20.4	5.3	**19.1**	**4.4**



Xie S , Girshick R , Dollár, Piotr, et al. Aggregated Residual Transformations for Deep Neural Networks, CVPR 2017.

ResNeXt

- ✓ **与GoogLeNet的区别（网络结构方面）**

  ResNeXt每个Block中拓扑结构完全相同，

  GoogLeNet每个Block中的拓扑结构不同

  GoogLeNet每个Block中的卷积核大小、数量等需要人为设置，对于新问题来说调参困难

- ✓ **参数量没有增长，计算量增长少**

  ResNet-50 参数量：25.5MB
  ResNeXt-50(32x4d) 参数量：25.0MB

stage	output	ResNet-50	**ResNeXt-50 (32×4d)**
conv1	112×112	7×7, 64, stride 2	7×7, 64, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128, C=32 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3	28×28	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256, C=32 \\ 1\times1, 512 \end{bmatrix}\times4$
conv4	14×14	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512, C=32 \\ 1\times1, 1024 \end{bmatrix}\times6$
conv5	7×7	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 1024 \\ 3\times3, 1024, C=32 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params.		$\mathbf{25.5}\times10^6$	$\mathbf{25.0}\times10^6$
FLOPs		$\mathbf{4.1}\times10^9$	$\mathbf{4.2}\times10^9$

SE-Net

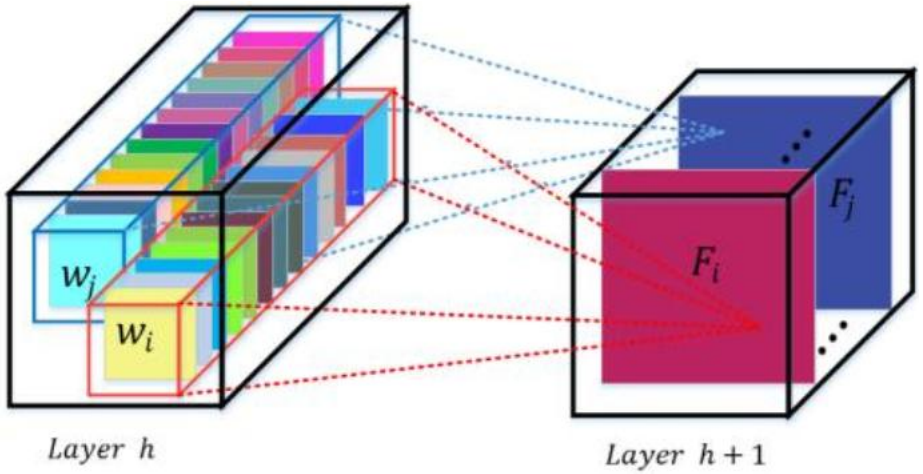
最后一届ImageNet的冠军模型！

✓ **引入了注意力机制**

显示的建模特征通道之间的相互依赖关系

通过学习的方式获取每个通道的权重（重要程度）。

根据权重去提升有用的特征，并抑制对当前任务用处不大的特征，从而提升网络的表示能力



	224 × 224		320 × 320 / 299 × 299	
	top-1 err.	top-5 err.	top-1 err.	top-5 err.
ResNet-152 [10]	23.0	6.7	21.3	5.5
ResNet-200 [11]	21.7	5.8	20.1	4.8
Inception-v3 [44]	-	-	21.2	5.6
Inception-v4 [42]	-	-	20.0	5.0
Inception-ResNet-v2 [42]	-	-	19.9	4.9
ResNeXt-101 (64 × 4d) [47]	20.4	5.3	19.1	4.4
DenseNet-264 [14]	22.15	6.12	-	-
Attention-92 [46]	-	-	19.5	4.8
Very Deep PolyNet [51] †	-	-	18.71	4.25
PyramidNet-200 [8]	20.1	5.4	19.2	4.7
DPN-131 [5]	19.93	5.12	18.55	4.16
SENet-154	**18.68**	**4.47**	**17.28**	**3.79**
NASNet-A (6@4032) [55] †	-	-	17.3‡	3.8‡
SENet-154 (post-challenge)	-	-	**16.88‡**	**3.58‡**

Hu J , Shen L , Albanie S , et al. Squeeze-and-Excitation Networks. IEEE TPAMI, 2017.

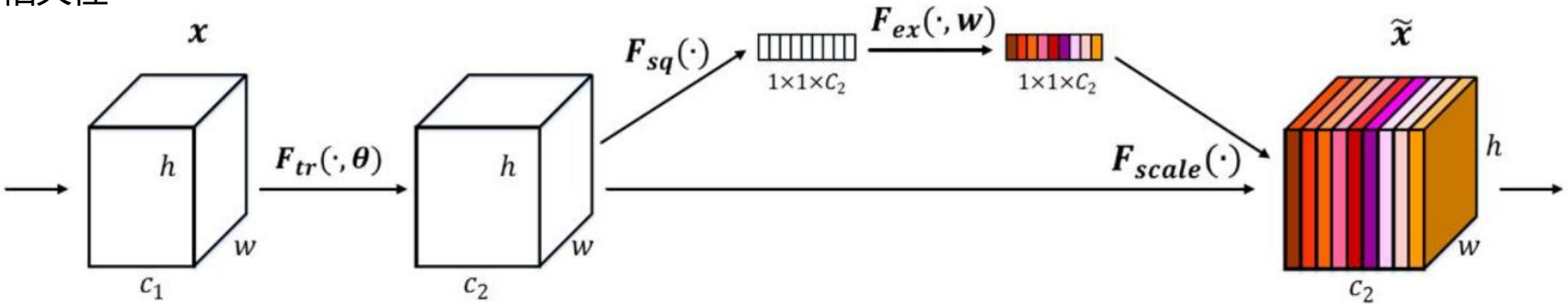
SE-Net

✓ **Squeeze和Excitation操作**

Squeeze用于编码全局信息，它使用全局平均池化来压缩特征图，输出一个C维向量，来表示特征通道响应的全局分布

Excitation用于为每个通道生成权重，从而建模特征通道之间的相关性

将输入的权重和原始特征图相乘，完成在通道维度对原始特征的重标定！

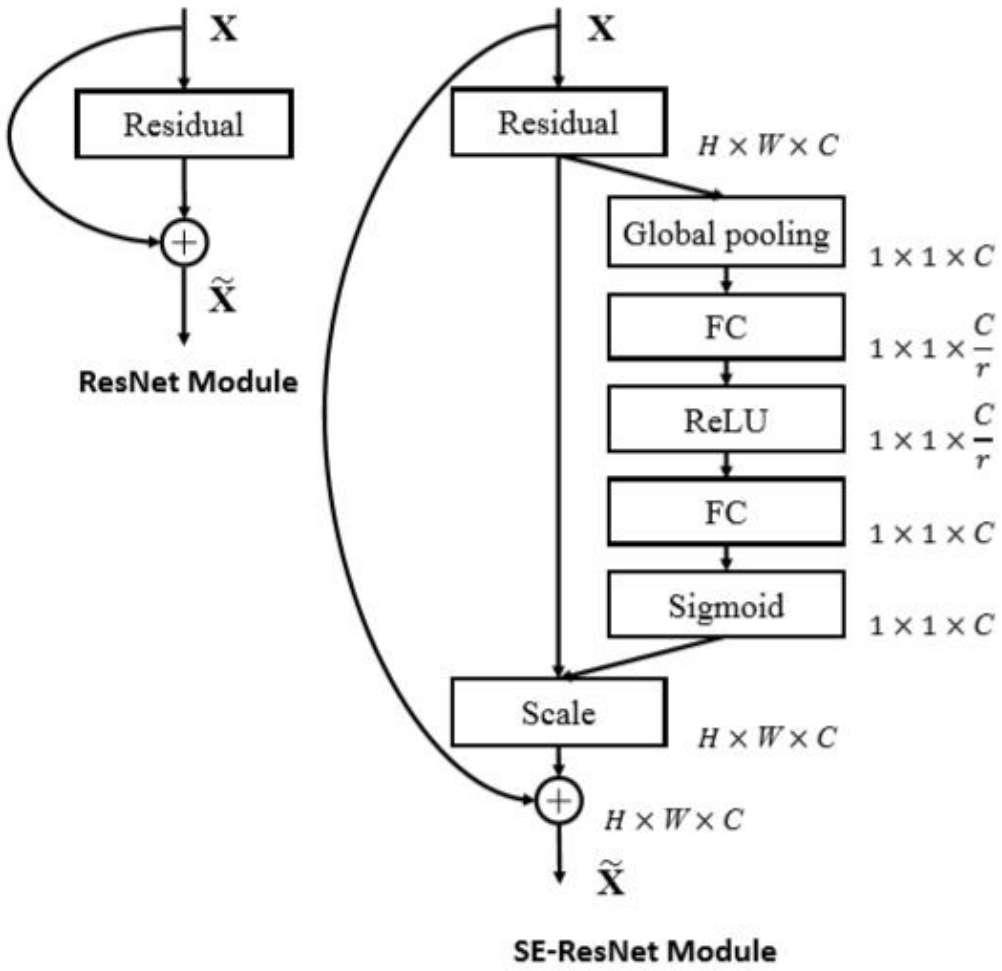


SE-Net

✓ **复杂度低，易于部署**

SE-ResNet-50 vs. ResNet-50

- Parameters: 2%~10% additional parameters
- Computation cost: <1% additional computation
- GPU inference time: 10% additional time
- CPU inference time: <2% additional time



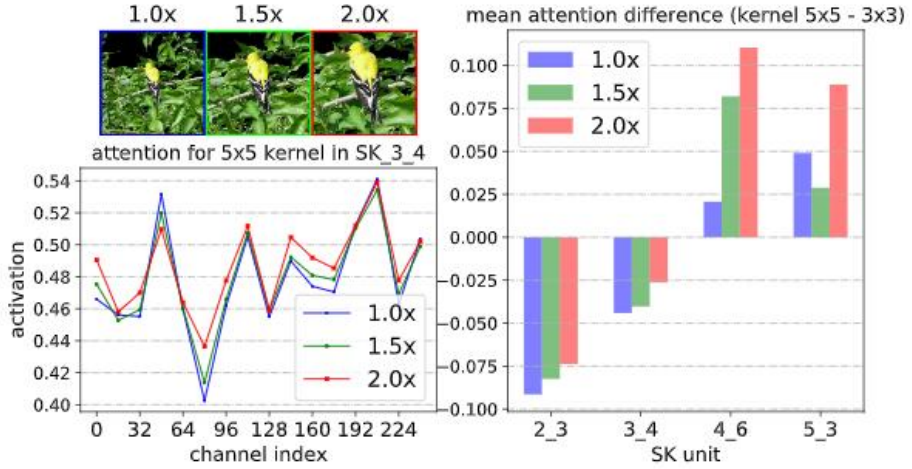
SK-Net

受皮质神经元根据不同的刺激可动态调节其自身的感受野启发！

✓ **引入了动态选择机制**

使得每一个神经元可以根据多尺度的输入信息来动态的调整其感受野的大小

使用不同大小的卷积核提取特征，再对多个分支的信息进行融合



	top-1 err (%)		#P	GFLOPs
	224×	320×		
ResNeXt-50	22.23	21.05	25.0M	4.24
AttentionNeXt-56 [44]	21.76	–	31.9M	6.32
InceptionV3 [43]	–	21.20	27.1M	5.73
ResNeXt-50 + BAM [32]	21.70	20.15	25.4M	4.31
ResNeXt-50 + CBAM [45]	21.40	20.38	27.7M	4.25
SENet-50 [12]	21.12	19.71	27.7M	4.25
SKNet-50 (ours)	**20.79**	**19.32**	27.5M	4.47
ResNeXt-101	21.11	19.86	44.3M	7.99
Attention-92 [44]	–	19.50	51.3M	10.43
DPN-92 [5]	20.70	19.30	37.7M	6.50
DPN-98 [5]	20.20	18.90	61.6M	11.70
InceptionV4 [41]	–	20.00	42.0M	12.31
Inception-ResNetV2 [41]	–	19.90	55.0M	13.22
ResNeXt-101 + BAM [32]	20.67	19.15	44.6M	8.05
ResNeXt-101 + CBAM [45]	20.60	19.42	49.2M	8.00
SENet-101 [12]	20.58	18.61	49.2M	8.00
SKNet-101 (ours)	**20.19**	**18.40**	48.9M	8.46

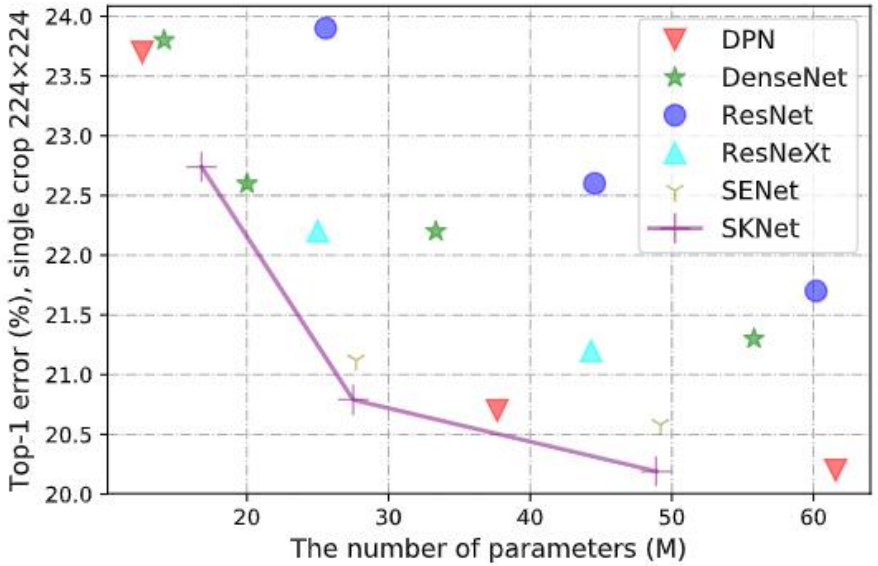
Li, Xiang, et al. Selective Kernel Networks. CVPR, 2019.

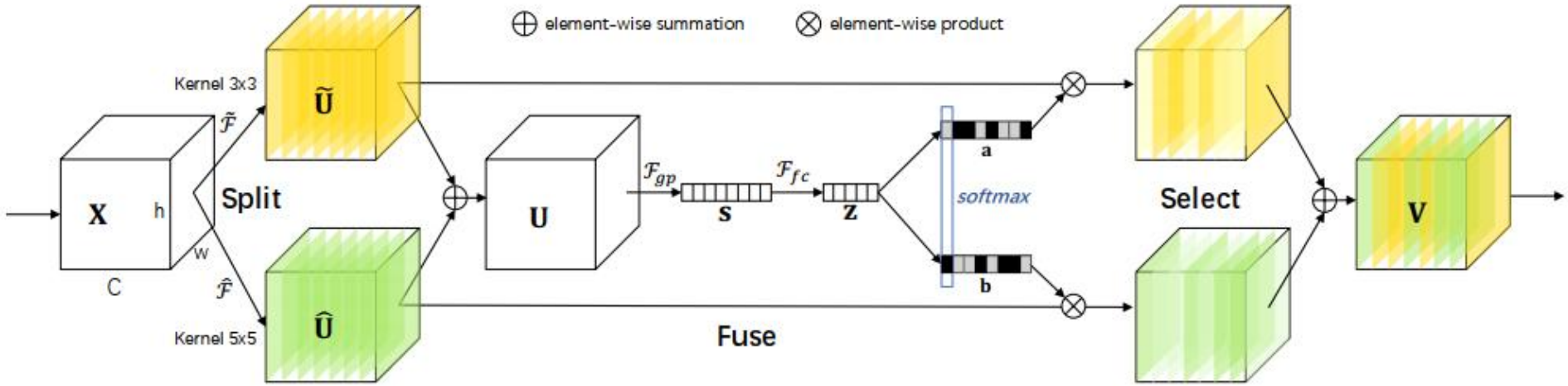
SK-Net

Split操作构建多路子分支，每个分支对应的卷积核大小不同，对应不同尺度的感受野

Fuse操作用于合并各路分支的信息，生成全局的互补信息表示

Select操作根据特征表示预测出注意力权重，并使用这个权重来聚合不同尺度的特征图



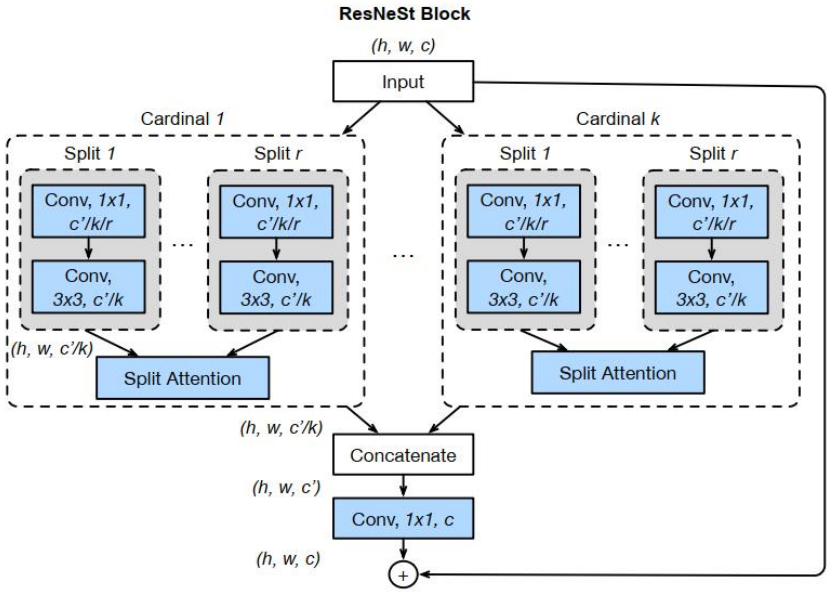


ResNeSt

分类、检测、分割同时达到SOTA！

- ✓ **借鉴了ResNeXt和SK-Net的思想**

  将网络划分成多个相同的拓扑分支，同时在每个分支内将输入划分成多份，将每一份并行输入到网络中，再经过 Split-Attention操作融合特征



	#P	GFLOPs	top-1 acc (%)	
			224×	320×
ResNet-50 23	25.5M	4.14	76.15	76.86
ResNeXt-50 60	25.0M	4.24	77.77	78.95
SENet-50 29	27.7M	4.25	78.88	80.29
ResNetD-50 26	25.6M	4.34	79.15	79.70
SKNet-50 38	27.5M	4.47	79.21	80.68
ResNeSt-50-fast(ours)	27.5M	4.34	**80.64**	**81.43**
ResNeSt-50(ours)	27.5M	5.39	81.13	81.82
ResNet-101 23	44.5M	7.87	77.37	78.17
ResNeXt-101 60	44.3M	7.99	78.89	80.14
SENet-101 29	49.2M	8.00	79.42	81.39
ResNetD-101 26	44.6M	8.06	80.54	81.26
SKNet-101 38	48.9M	8.46	79.81	81.60
ResNeSt-101-fast(ours)	48.2M	8.07	**81.97**	**82.76**
ResNeSt-101(ours)	48.3M	10.2	82.27	83.00

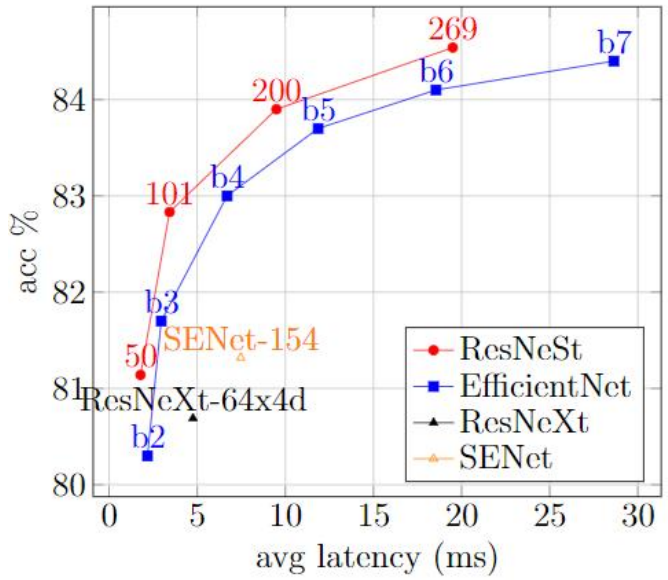
Zhang, Hang, et al. ResNeSt: Split-Attention Networks. ArXiv:2004.08955, 2020.

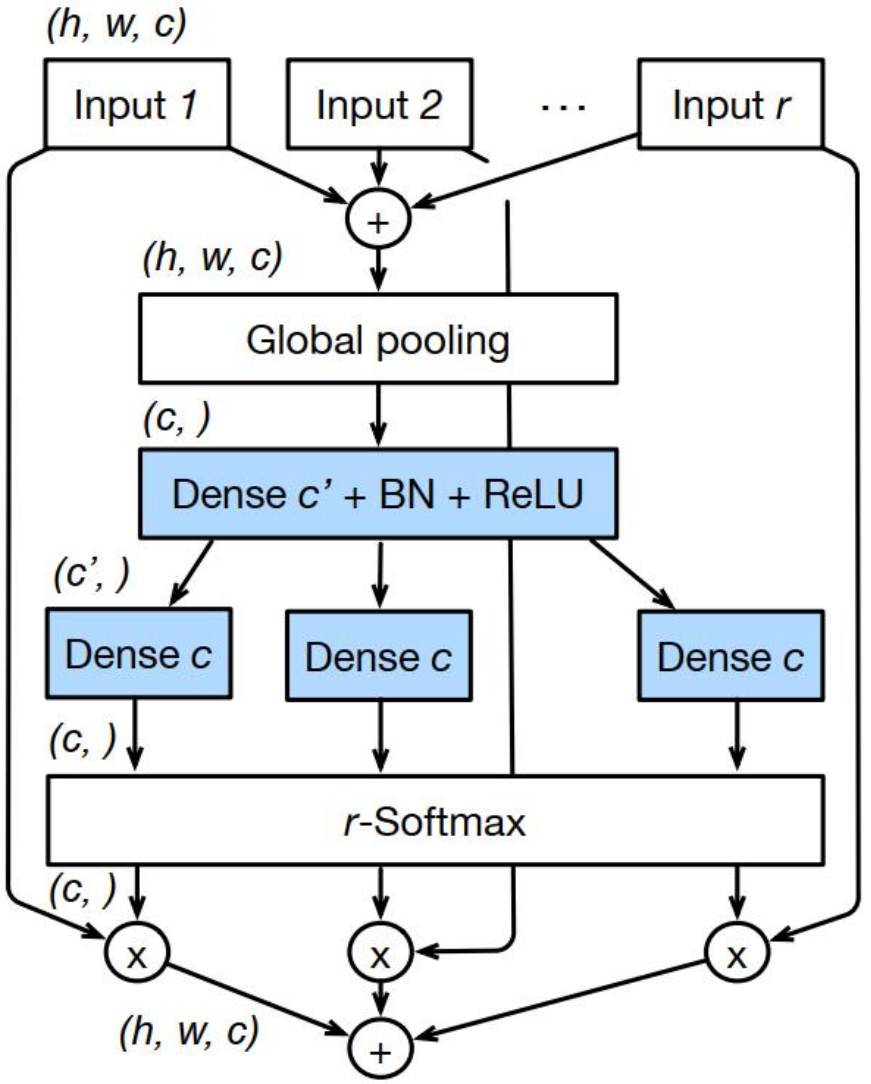
ResNeSt

紧跟前沿，积极思考并快速Follow

✓ Split-Attention操作

首先将输入划分成多份。然后对输入进行全局平均池化操作，获得对每个通道编码后的全局上下文信息。再对每一份输入的全局上下文信息单独进行处理，获取每个通道的注意力权重。最后使用注意力权重和输入相乘，获得提升后的特征表示



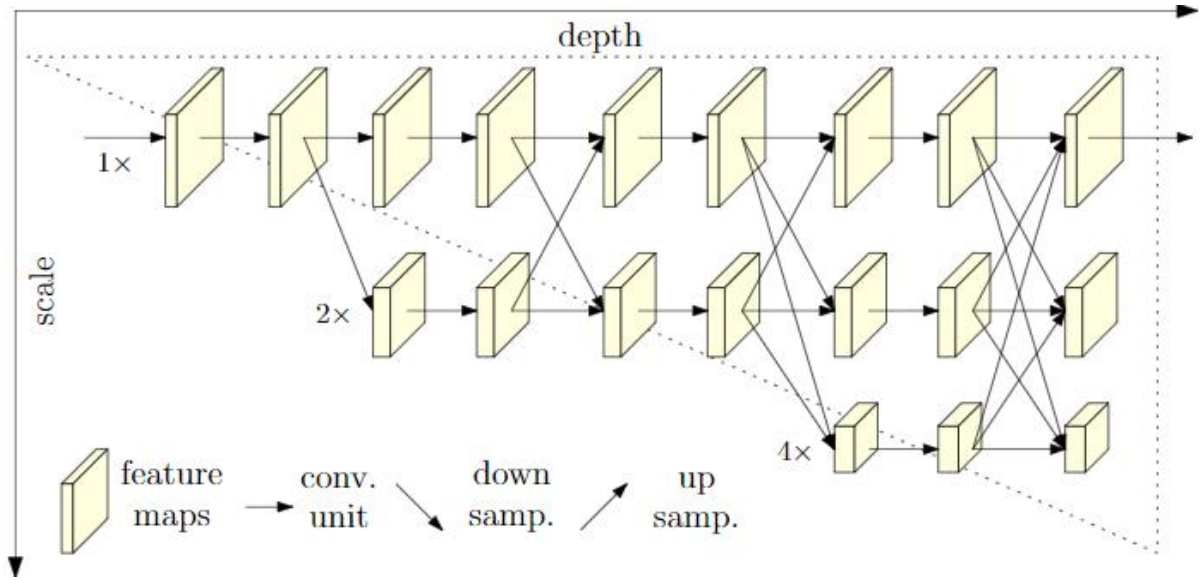


HR-Net

适用于姿势估计、语义分割、目标检测的高分辨率网络!

- ✓ **高分辨率的特征表示**

  通过并行连接高分辨率与低分辨率的特征图，整个网络始终维持高分辨率的特征表示，使得输出特征图的语义信息更加丰富且具有位置敏感性。



	backbone	mIoU	iIoU cla.	IoU cat.	iIoU cat.
Model learned on the `train` set					
PSPNet [181]	D-ResNet-101	78.4	56.7	90.6	78.6
PSANet [182]	D-ResNet-101	78.6	-	-	-
PAN [82]	D-ResNet-101	78.6	-	-	-
AAF [69]	D-ResNet-101	79.1	-	-	-
HRNetV2	HRNetV2-W48	**80.4**	**59.2**	**91.5**	**80.8**
Model learned on the `train+val` set					
GridNet [42]	-	69.5	44.1	87.9	71.1
LRR-4x [46]	-	69.7	48.0	88.2	74.7
DeepLab [19]	D-ResNet-101	70.4	42.6	86.4	67.7
LC [84]	-	71.1	-	-	-
Piecewise [91]	VGG-16	71.6	51.7	87.3	74.1
FRRN [114]	-	71.8	45.5	88.9	75.1
RefineNet [90]	ResNet-101	73.6	47.2	87.9	70.6
PEARL [65]	D-ResNet-101	75.4	51.6	89.2	75.1
DSSPN [88]	D-ResNet-101	76.6	56.2	89.6	77.8
LKM [111]	ResNet-152	76.9	-	-	-
DUC-HDC [144]	-	77.6	53.6	90.1	75.2
SAC [176]	D-ResNet-101	78.1	-	-	-
DepthSeg [73]	D-ResNet-101	78.2	-	-	-
ResNet38 [151]	WResNet-38	78.4	59.1	90.9	78.1
BiSeNet [166]	ResNet-101	78.9	-	-	-
DFN [167]	ResNet-101	79.3	-	-	-
PSANet [182]	D-ResNet-101	80.1	-	-	-
PADNet [159]	D-ResNet-101	80.3	58.8	90.8	78.5
CFNet [173]	D-ResNet-101	79.6	-	-	-
Auto-DeepLab [95]	-	80.4	-	-	-
DenseASPP [181]	WDenseNet-161	80.6	59.1	90.9	78.1
SVCNet [33]	ResNet-101	81.0	-	-	-
ANN [195]	D-ResNet-101	81.3	-	-	-
CCNet [61]	D-ResNet-101	81.4	-	-	-
DANet [43]	D-ResNet-101	81.5	-	-	-
HRNetV2	HRNetV2-W48	81.6	**61.8**	**92.1**	**82.2**
HRNetV2 + OCR [170]	HRNetV2-W48	**82.5**	61.7	**92.1**	81.6

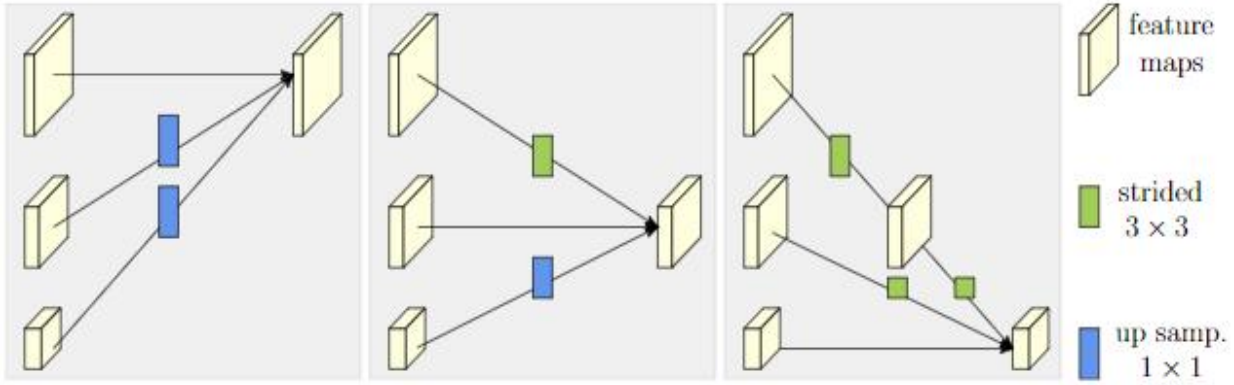
Cityscapes

Sun, Ke , et al, Deep High-Resolution Representation Learning for Human Pose Estimation, CVPR 2019.

HR-Net

- ✓ **多尺度特征融合**

引入了Exchange-Unit，通过在多个不同分辨率的子网络之间重复交换信息，来融合不同分辨率的特征图（通过3*3卷积和上采样模块实现，即右图的绿色和蓝色部分）



Method	#Params	GFLOPs	PCKh@0.5
Insafutdinov et al. [27]	42.6M	41.2	88.5
Newell et al. [40]	25.1M	19.1	90.9
Yang et al. [77]	28.1M	21.3	92.0
Tang et al. [62]	15.5M	15.6	92.3
SimpleBaseline [72]	68.6M	20.9	91.5
HRNet-W32	28.5M	9.5	92.3





RepVGG

Making VGG-style ConvNets Great Again!

- ✓ 现在的网络大都引入了残差结构来提高网络的性能，对内存消耗不友好。因为各个分支的结果需要保存，直到最后一步，才能把各分支显存释放掉。
- ✓ 3x3卷积：现在计算库和硬件对3x3的卷积有速度优化。
- ✓ 推理的时候合并分支，减少需要的内存和计算量。性能和速度上直达SOTA

重参化！

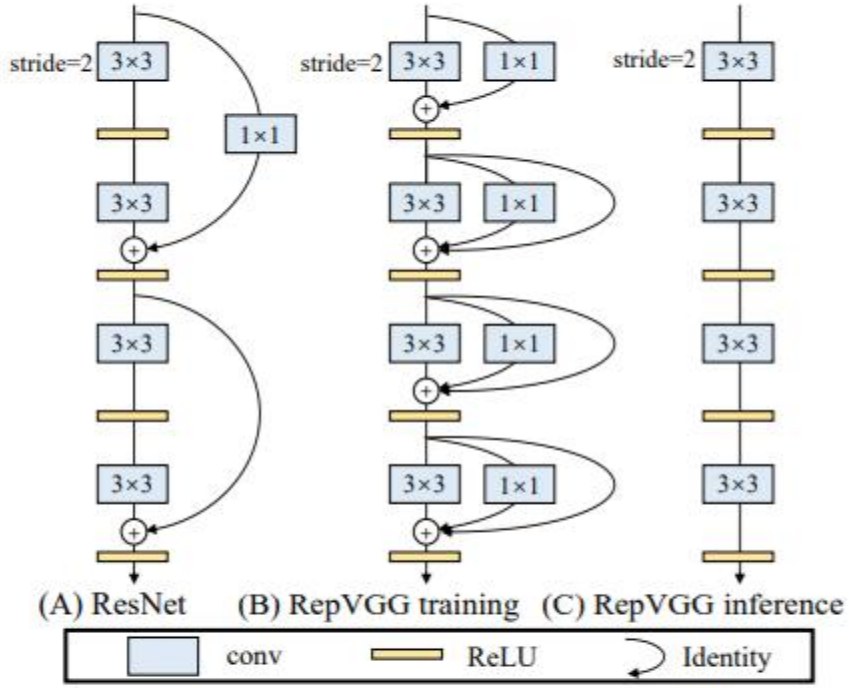


Figure 2: Sketch of RepVGG architecture. RepVGG has 5 stages and conducts down-sampling via stride-2 convolution at the beginning of a stage. Here we only show the first 4 layers of a specific stage. As inspired by ResNet [12], we also use identity and 1×1 branches, but only for training.

Xiaohan Ding, Xiangyu Zhang et al., RepVGG: Making VGG-style ConvNets Great Again, CVPR 2021.

RepVGG

训练多路模型，推理时通过重参数化合并成单路！

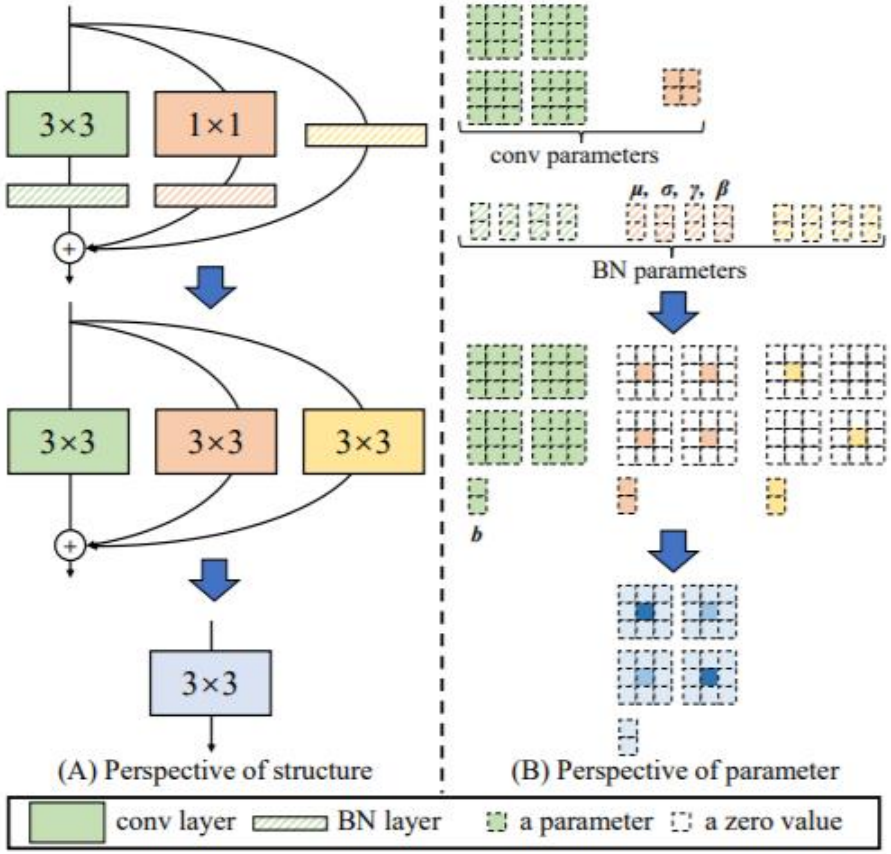


Figure 4: Structural re-parameterization of a RepVGG block. For the ease of visualization, we assume $C_2 = C_1 = 2$, thus the 3×3 layer has four 3×3 matrices and the kernel of 1×1 layer is a 2×2 matrix.

1. BN层的合并

$$\begin{aligned} \mathrm{M}^{(2)} &= \mathrm{bn}(\mathrm{M}^{(1)} * \mathrm{W}^{(3)}, \boldsymbol{\mu}^{(3)}, \boldsymbol{\sigma}^{(3)}, \boldsymbol{\gamma}^{(3)}, \boldsymbol{\beta}^{(3)}) \\ &+ \mathrm{bn}(\mathrm{M}^{(1)} * \mathrm{W}^{(1)}, \boldsymbol{\mu}^{(1)}, \boldsymbol{\sigma}^{(1)}, \boldsymbol{\gamma}^{(1)}, \boldsymbol{\beta}^{(1)}) \\ &+ \mathrm{bn}(\mathrm{M}^{(1)}, \boldsymbol{\mu}^{(0)}, \boldsymbol{\sigma}^{(0)}, \boldsymbol{\gamma}^{(0)}, \boldsymbol{\beta}^{(0)}) . \end{aligned} \tag{1}$$

Otherwise, we simply use no identity branch, hence the above equation only has the first two terms. Here bn is the inference-time BN function, formally, $\forall 1 \leq i \leq C_2$,

$$\mathrm{bn}(\mathrm{M}, \boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\gamma}, \boldsymbol{\beta})_{:,i,:,:} = (\mathrm{M}_{:,i,:,:} - \boldsymbol{\mu}_i)\frac{\boldsymbol{\gamma}_i}{\boldsymbol{\sigma}_i} + \boldsymbol{\beta}_i . \tag{2}$$

We first convert every BN and its preceding conv layer into a conv with a bias vector. Let $\{\mathrm{W}', \mathbf{b}'\}$ be the kernel and bias converted from $\{\mathrm{W}, \boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\gamma}, \boldsymbol{\beta}\}$, we have

$$\mathrm{W}'_{i,:,:,:} = \frac{\boldsymbol{\gamma}_i}{\boldsymbol{\sigma}_i}\mathrm{W}_{i,:,:,:} , \quad \mathbf{b}'_i = -\frac{\boldsymbol{\mu}_i \boldsymbol{\gamma}_i}{\boldsymbol{\sigma}_i} + \boldsymbol{\beta}_i . \tag{3}$$

Then it is easy to verify that $\forall 1 \leq i \leq C_2$,

$$\mathrm{bn}(\mathrm{M} * \mathrm{W}, \boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\gamma}, \boldsymbol{\beta})_{:,i,:,:} = (\mathrm{M} * \mathrm{W}')_{:,i,:,:} + \mathbf{b}'_i . \tag{4}$$

RepVGG

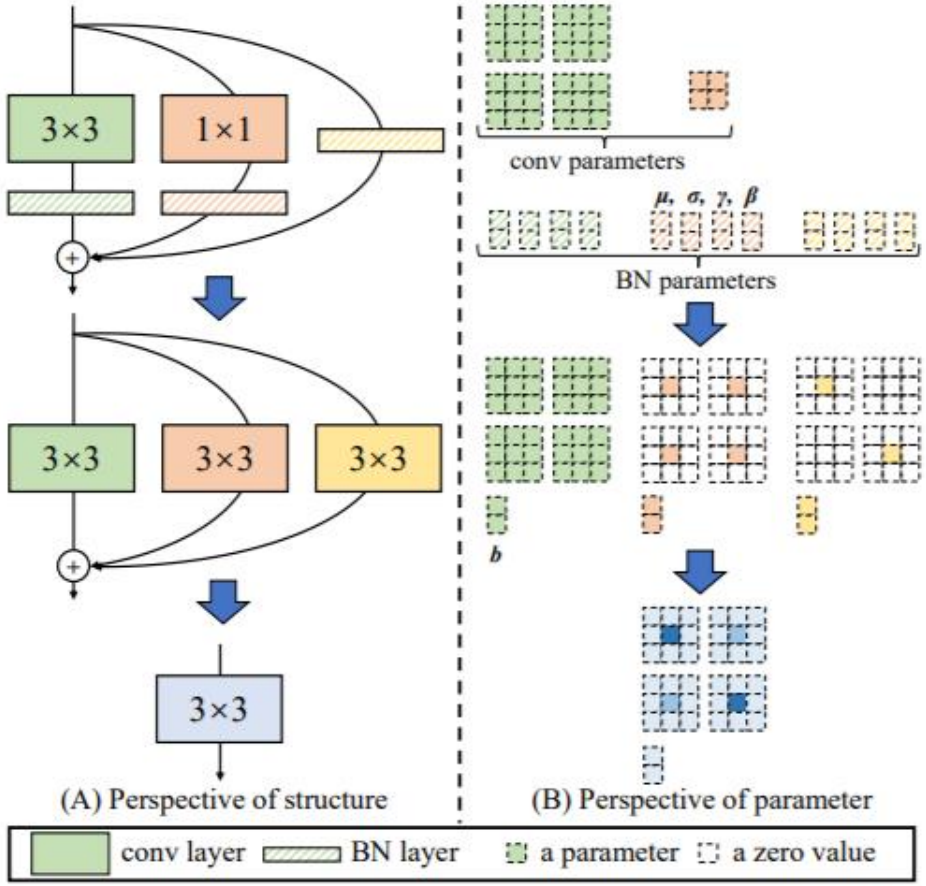


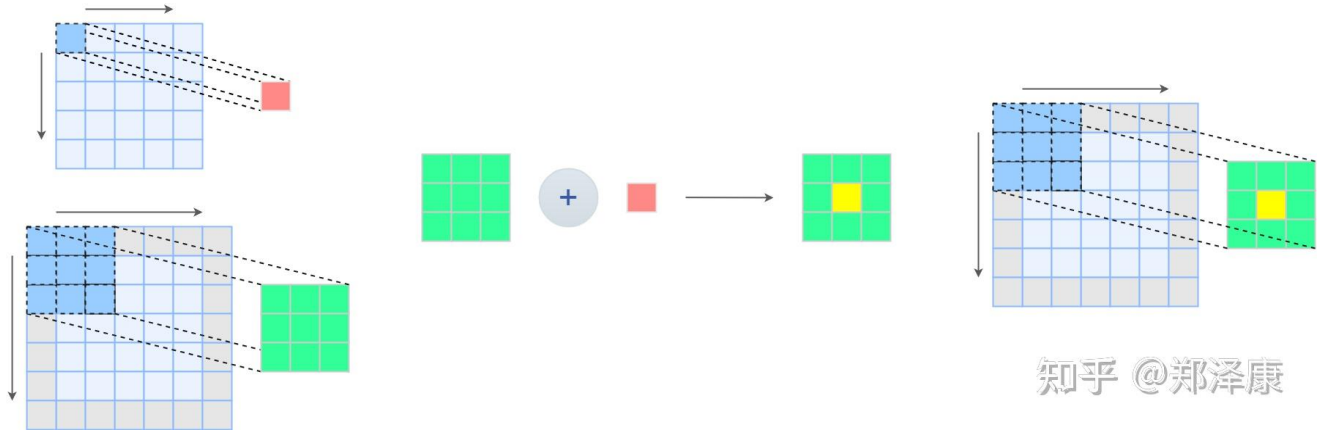
Figure 4: Structural re-parameterization of a RepVGG block. For the ease of visualization, we assume $C_2 = C_1 = 2$, thus the 3×3 layer has four 3×3 matrices and the kernel of 1×1 layer is a 2×2 matrix.

2. 将1x1的卷积转换成3x3的卷积

将1x1的卷积padding为3x3的卷积，这样这个3x3的卷积的移动范围就和1x1的卷积一样了

3. 将Identity分支转换成3x3的卷积

将Identity分支变为1x1的卷积，对应的通道的卷积核值为1，其他为0，再padding为3x3的卷积



知乎 @郑泽康

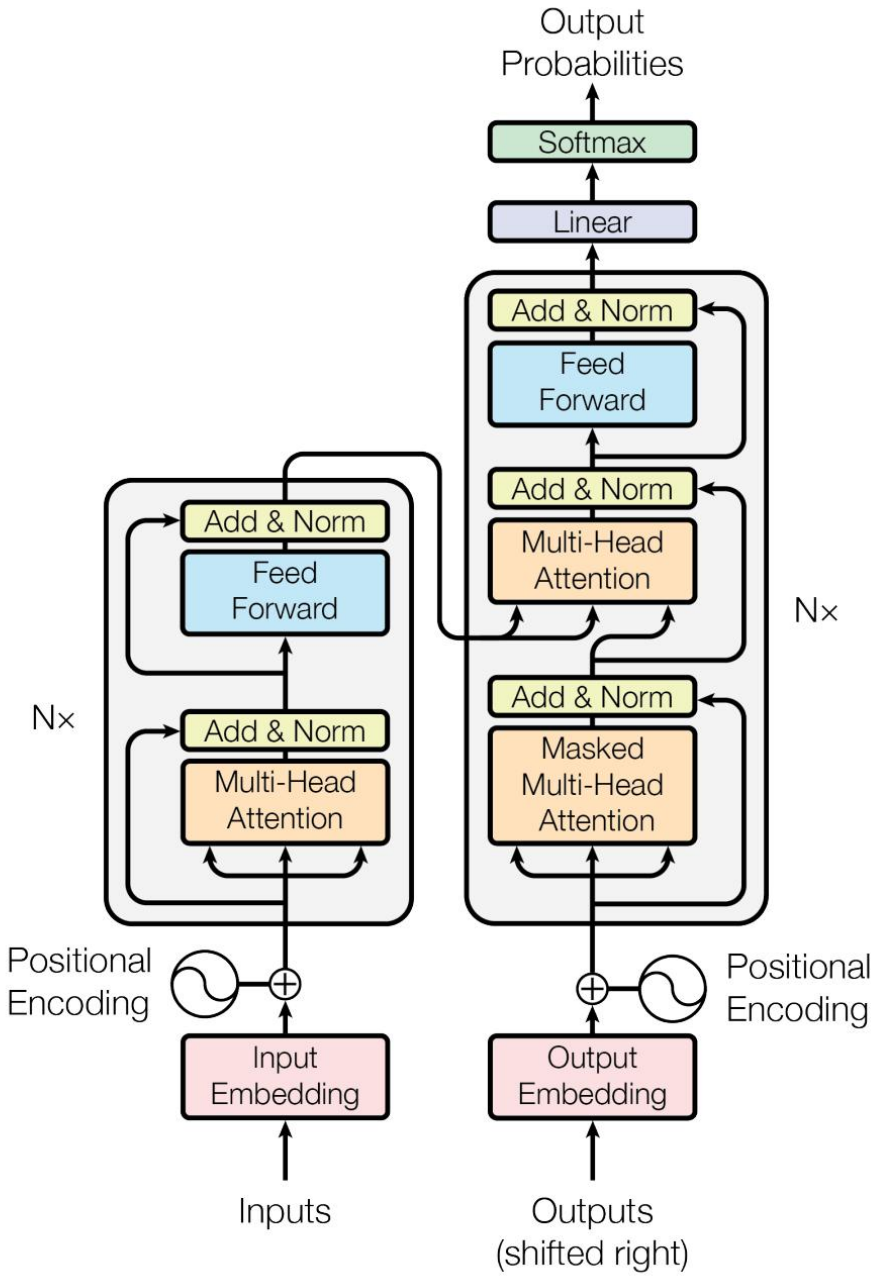
Part 3

Transformer及典型网络架构

Transformer回顾

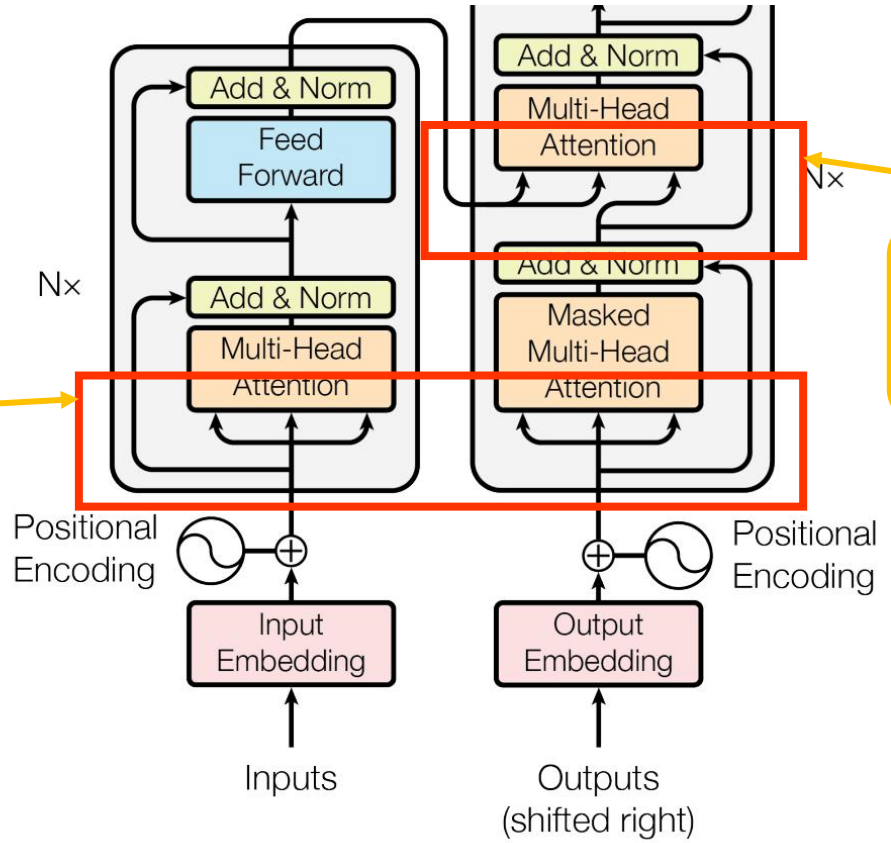
模型框架

- ◆ 解码器多头注意力层融合编、解码器信息:
  - ✓ 解码器的多头注意力层输入包括解码器中上一层的输出和编码器的输出
- ◆ Mask防止训练时使用到未来输出的单词:
  - ✓ 保证预测位置i的信息只会用到i之前的输出
  - ✓ 解码器是像RNN那样一个一个进行预测的
- ◆ 前馈连接层FFN就是两层全连接:
  - ✓ 使用非线性函数拟合数据，并进行维度变换
- ◆ 层归一化:
  - ✓ 输入维度(batchsize, seq_len, embedding)
  - ✓ 对batch中每个样本，在seq_len上进行归一化



Transformer回顾

注意力机制



QKV均为上一层输出的编码
故称**自**注意力

Q为上一层输出的编码
KV为编码器的输出，计算**交叉**注意力

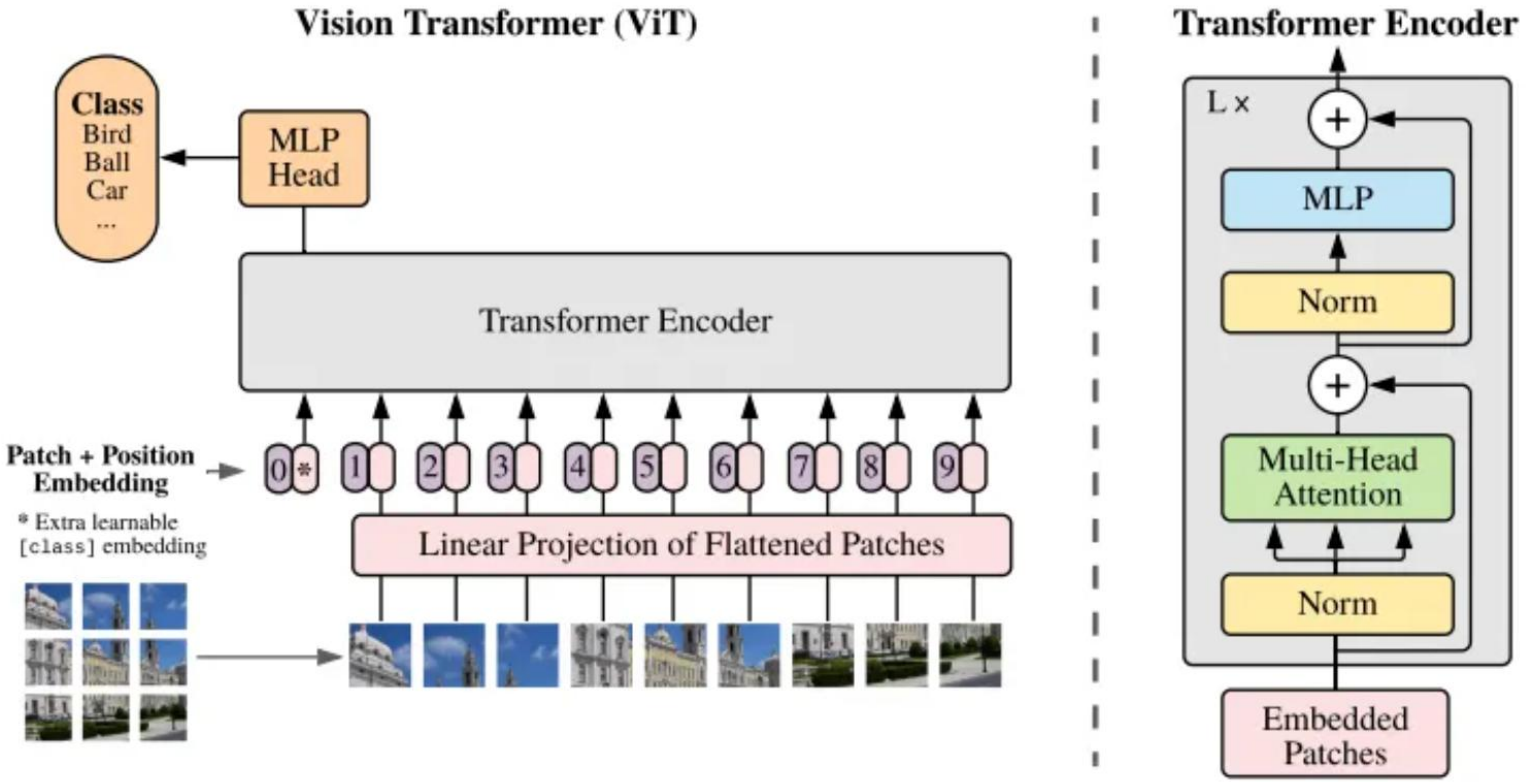
主干网络：ViT

模型框架

将输入图片分为多个16x16的patch，再将每个patch投影为固定长度的向量送入Transformer的编码器。

对于224x224大小的图片，这样得到14x14个（196个）token。

……将一个视觉问题转化成seq2seq问题



ViT只用了原始Transformer的编码器

主干网络：ViT

模型细节

- 类别编码:
  - ✓ 为了对图片分类，因此在输入序列中加入一个特殊的token[cls]，该token对应的输出即为最后的类别预测
  - ✓ 对于224x224大小的图片，总共有196+1=197个token
- 位置编码:
  - ✓ Transformer的位置编码可以用固定算法生成，也可以学习，ViT通过学习生成位置编码
  - ✓ 位置编码直接加在patch embedding上，注意不是concat
- 预训练主干网络:
  - ✓ 通常在一个很大的数据集上预训练ViT，然后在下游任务相对小的数据集上微调

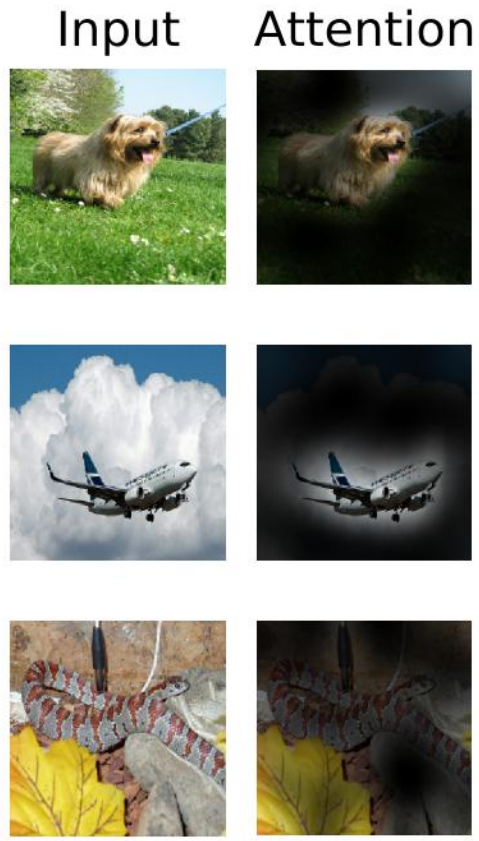


Figure 6: Representative examples of attention from the output token to the input space. See Appendix D.7 for details.

主干网络：Swin Transformer

滑动窗口机制和层级下采样

◆ 将Transformer从NLP应用到CV的挑战:

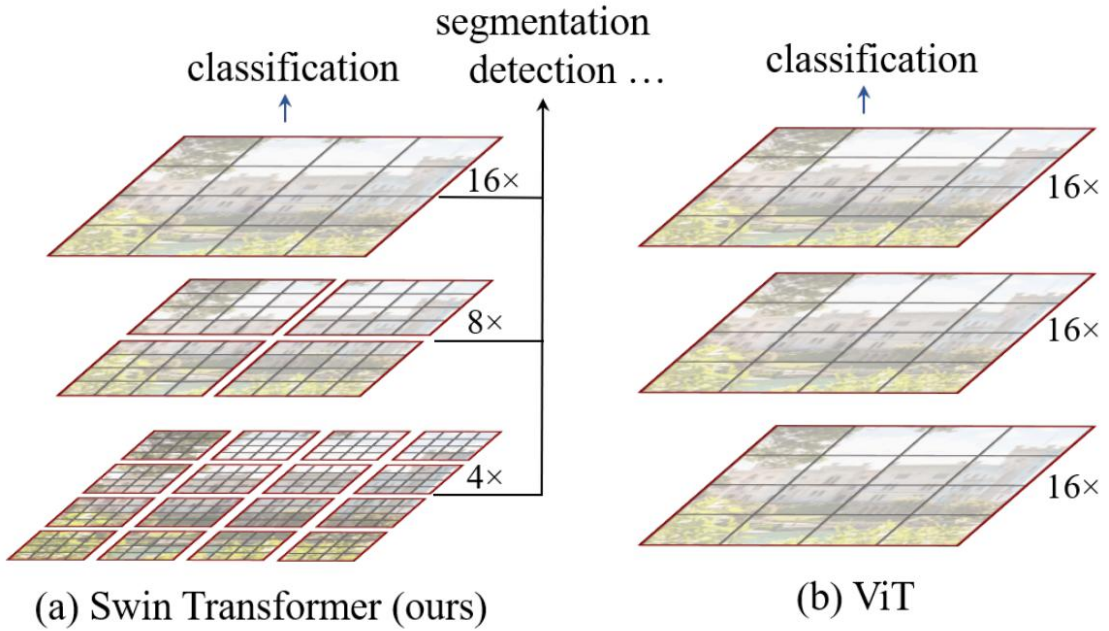
✓ 不同于NLP中一个单词（token）就是一个基本实体，CV中视觉实体尺度各不相同，用固定大小的token表示效果不好

✓ 图像分辨率高，像素点多，使得ViT模型自注意力的计算量与像素的平方成正比

◆ 滑动窗口机制:

✓ 包括不重叠的 local window和重叠的 cross-window

✓ 将注意力计算限制在一个窗口中，引入 CNN 卷积操作的局部性，且能大幅度节省计算量



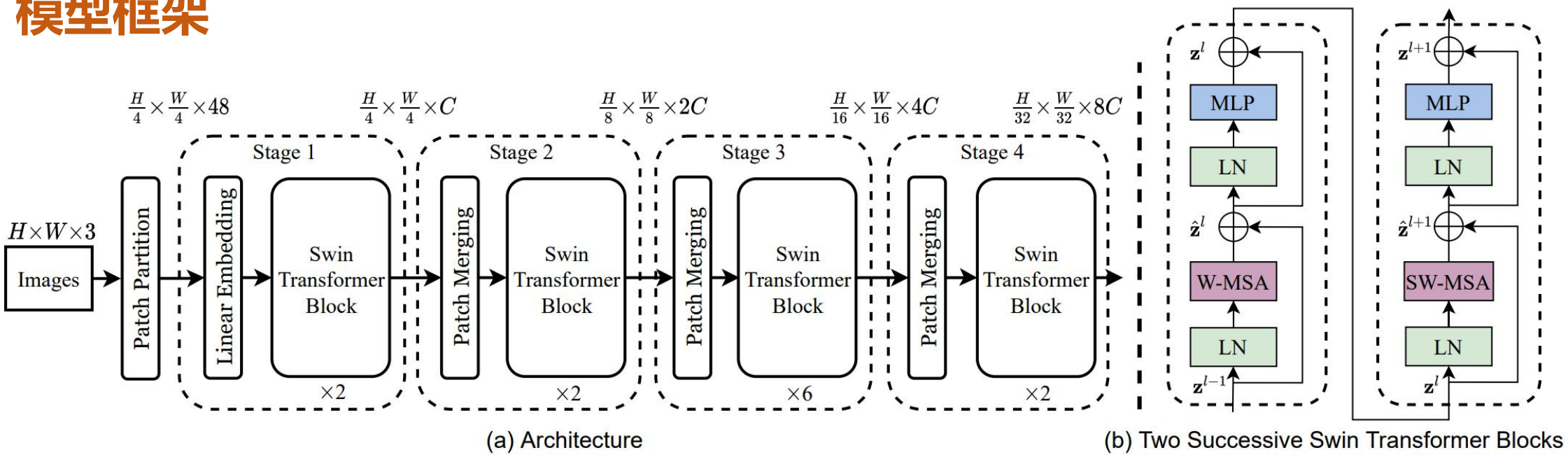
◆ 下采样层级设计:

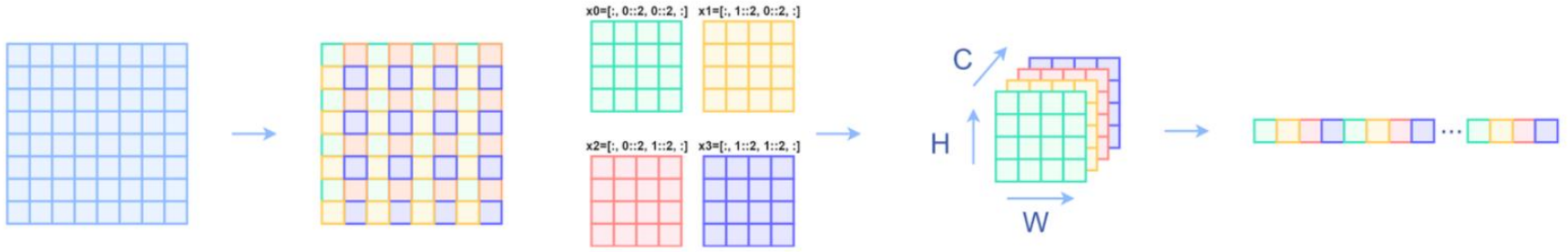
✓ 逐渐增大感受野，从而注意到全局的特征

只和窗口数量成线性关系

主干网络： Swin Transformer

模型框架

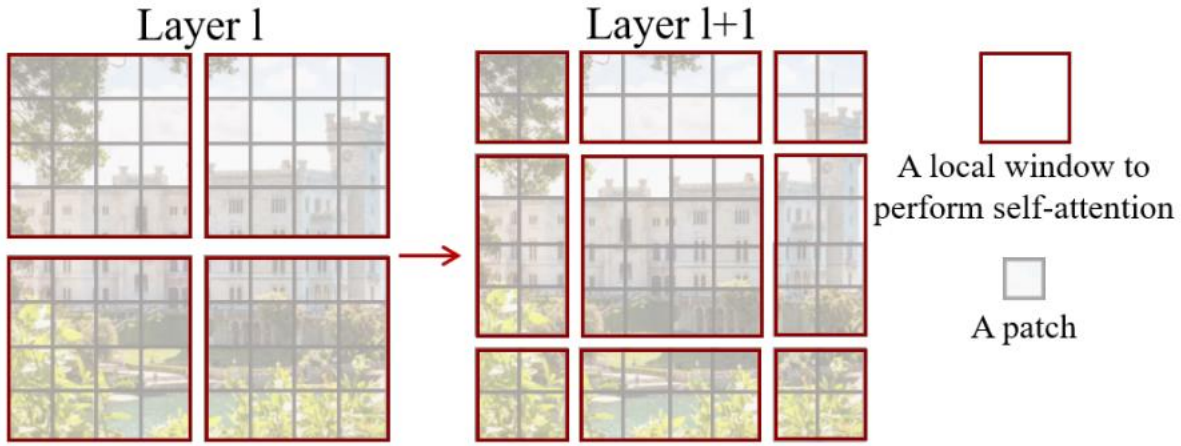




Patch Merging：
在每个Stage开始前做降采样，用于缩小分辨率，调整通道数进而形成层次化的设计，同时也能节省一定运算量

主干网络： Swin Transformer

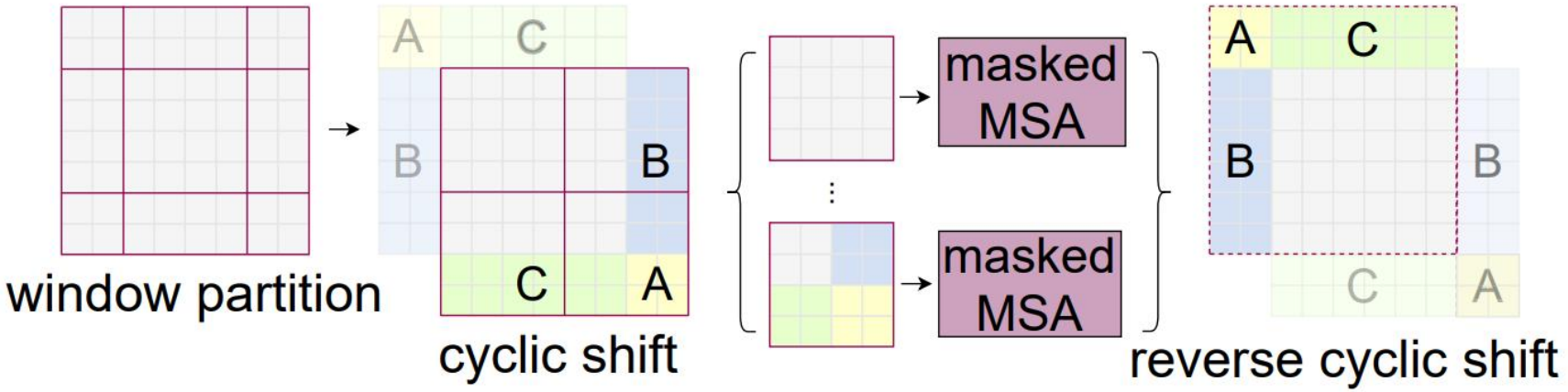
移位窗口注意力



◆ Shifted Window Attention:

✓ Swin Transformer在每个窗口下计算注意力，为了更好地和其他窗口进行信息交互，引入shifted window操作

✓ 引入了一个新问题：窗口的个数翻倍了，由原本的4个变成了 9 个



◆ 特征图移位操作:

✓ 通过对特征图移位，并给attention设置mask来间接解决上述问题。能在保持原有的 window 个数下，最后的计算结果等价

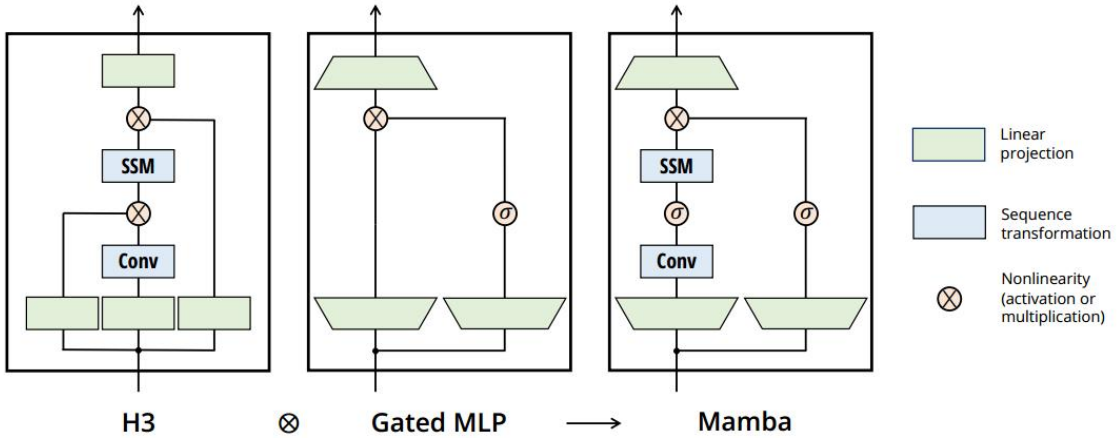
主干网络： Mamba

模型框架

Mamba是近期热门的模型之一。Mamba模型通过增强长距离依赖性和自适应融合机制，提高了复杂场景下的特征表达能力，适用于多种计算机视觉任务，特别是在捕捉细微和全局信息方面表现突出。

◆ Mamba block:

✓ Mamba模型是通过堆叠多个mamba block而来，是一种结合了H3块和Gated MLP块特点的简化设计习，并在设计中引入了SSM和现代激活函数，从而增强了模型的表现能力，最重要的部分是SSM状态空间模型。

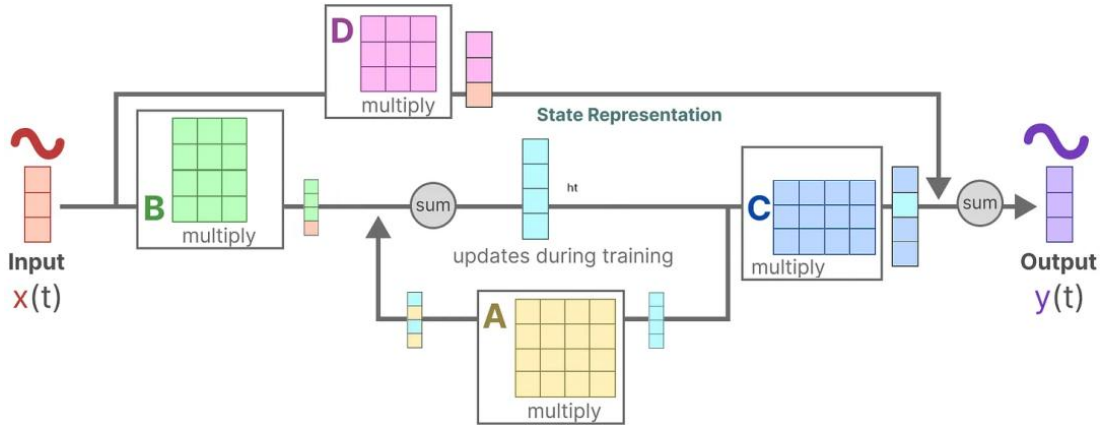


主干网络：Mamba

模型框架

- 状态转移方程:

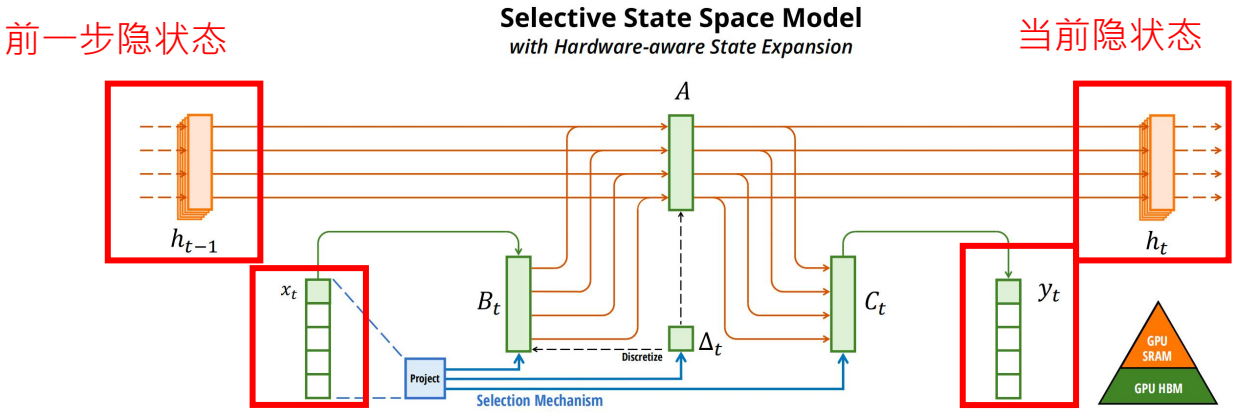
✓ 输出伴随着输入变化而变化，模型会存储每次处理的内部状态（上下文状态），同时会利用四个关键的矩阵完成内部状态的切换和控制输出。这四个矩阵就是该模型的参数



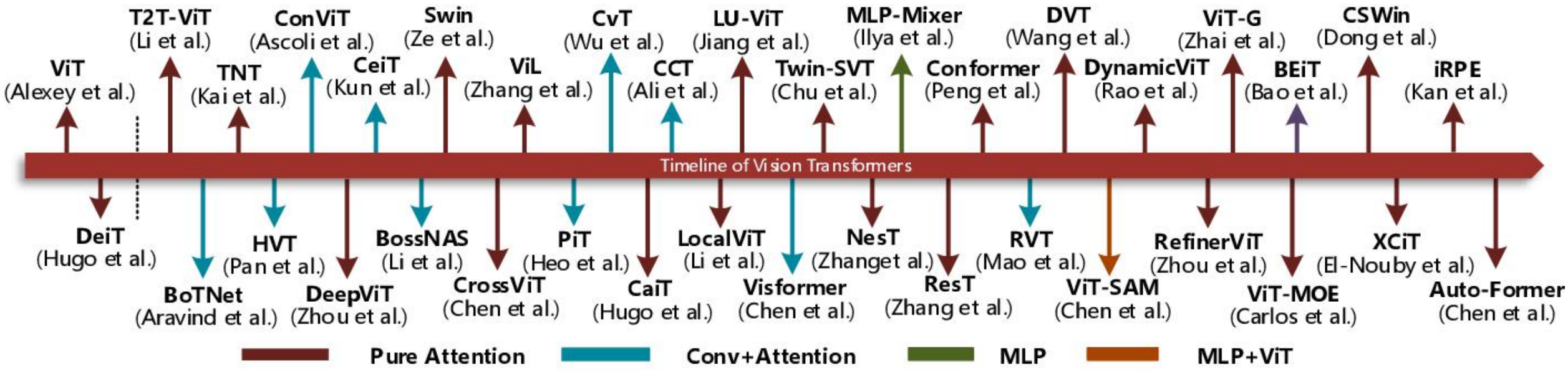
状态转移方程的示意图：中间x为状态变量，A为状态矩阵，主要用于将上个状态xt-1转换为xt，B为输入矩阵，C为输出矩阵，D为前馈矩阵。

- SSM

✓ Mamba根据状态转移方程设计的SSM模块，且需要一个硬件感知算法来保证计算的高效性，该算法在循环结构下实现近似并行计算的方法并减少了硬件间反复IO的次数，显著提高了计算效率



视觉Transformer进展



2020

横向进展

面向不同任务

目标检测

Faster RCNN

Cascade RCNN

SSD

YOLO

Mask RCNN

VOVNet（最新）

…

图像分割

FCN

DeepLab系列

SegNet

U-Net

Mask RCNN

…

前述卷积神经网络或Transformer是其中的一部分

轻量级神经网络

相关技术

参数修剪和共享

低秩因子分解

紧凑卷积滤波器

知识蒸馏

神经网络搜索

典型结构

MobileNet系列

ShuffleNet系列

EfficientNet

GhostNet

…

模型小、资源消耗低，计算速度快、性能好
易于部署在移动端和嵌入式端

THANKS